MicroserviceBase

v. 2.0.0

Nguyen Huynh Tri Cuong

09.02.2026

Contents

Chapter 1

Introduction

1.1 Introduction

MicroserviceBase is a Python framework for building, managing, and orchestrating microservices with RabbitMQ-based communication.

It provides:

- **Microservice Framework**: Base classes for creating services with RPC and pub/sub patterns
- **Service Registry**: Dynamic service registration and discovery via RabbitMQ exchange
- **FastAPI Bridge**: REST API and WebSocket gateway connecting GUI to microservices
- **Manager GUI**: Electron and browser-based dashboard for monitoring and controlling services
- **Local Process Hub**: Process lifecycle management with graceful shutdown
- **Hexagonal Architecture**: Clean separation via ports and adapters pattern

1.1.1 Target Audience

This package is designed for developers who need to:

- Build microservices that communicate via RabbitMQ message broker
- Monitor and manage services through a graphical dashboard
- Manage local service processes with start, stop, and import capabilities
- Deploy a standalone desktop application for service management

1.1.2 Prerequisites

- Python 3.10 or higher
- RabbitMQ server (message broker)
- pika package (RabbitMQ client for Python)
- FastAPI and uvicorn (for the bridge)
- Node.js and npm (for GUI development)

The sources of **MicroserviceBase** are available in GitHub.

Information about how to install the **MicroserviceBase** can be found in the README.

Chapter 2

Description

2.1 Description

2.1.1 Overview

MicroserviceBase is a Python framework for building microservices that communicate through RabbitMQ, with a built-in GUI for monitoring, a local process hub for lifecycle management, and a service registry for dynamic discovery. The system follows hexagonal architecture principles, ensuring testability, maintainability, and extensibility.

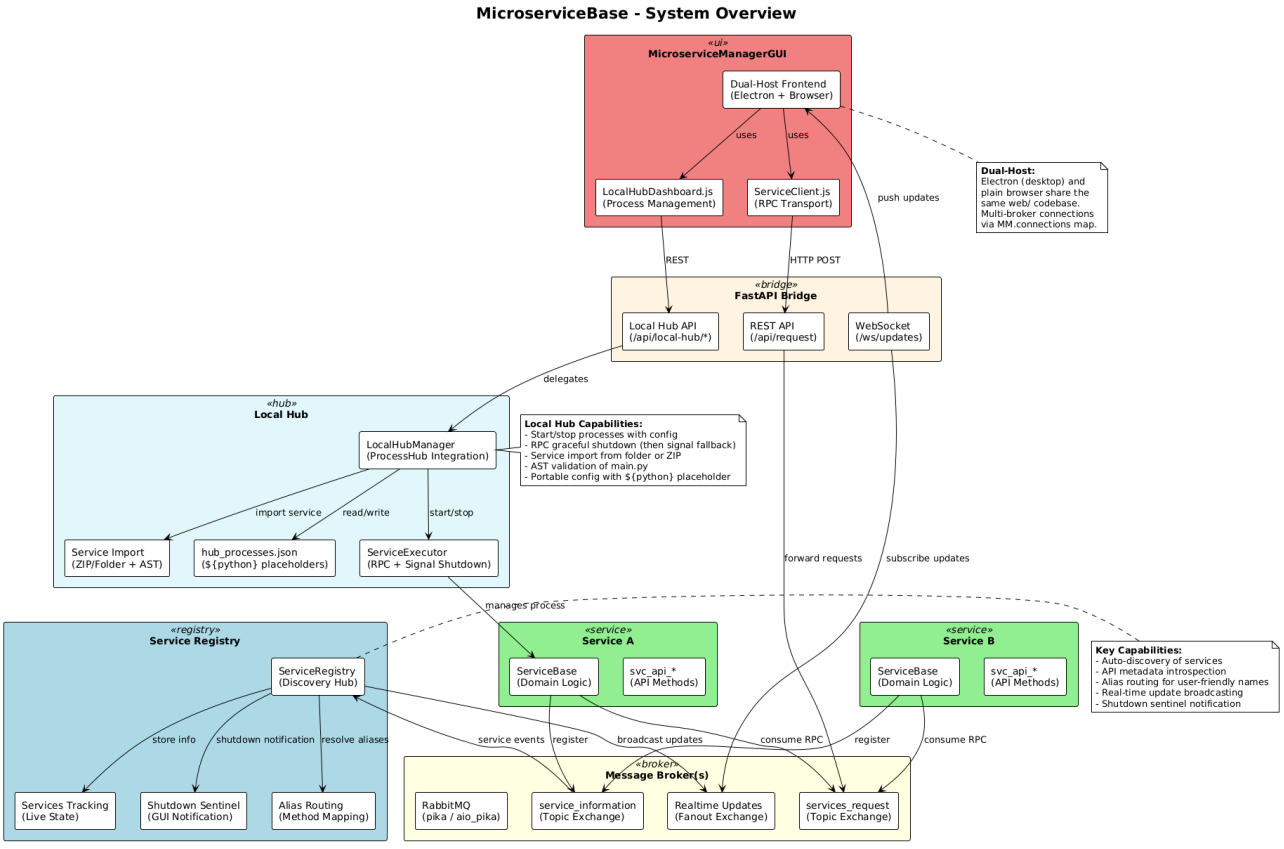


Figure 2.1: MicroserviceBase System Overview

Key Features

- **Microservice Framework**: Base classes for creating services with RPC and pub/sub patterns
- **Service Registry**: Dynamic service registration and discovery via RabbitMQ exchange
- **FastAPI Bridge**: REST API and WebSocket gateway connecting GUI to microservices
- **Manager GUI**: Electron + browser-based dashboard for monitoring and controlling services

- **Local Process Hub**: Start, stop, and manage service processes with graceful shutdown
- **Multi-Broker Support**: Connect to multiple RabbitMQ brokers simultaneously
- **Service Import**: Import microservice packages (folder or zip) into the local hub
- **Service Creator**: Interactive wizard for scaffolding new microservices
- **Standalone Installer**: Package as Windows desktop application (DevAtServGUI)

Design Philosophy

MicroserviceBase follows several key design principles:

1. **Hexagonal Architecture**: Domain logic is isolated from external concerns via ports and adapters
2. **Factory Pattern**: All components are created through factories, enabling easy swapping
3. **Dual Hosting**: GUI works in both Electron (desktop) and browser modes
4. **Graceful Shutdown**: Proper process cleanup on Windows using CTRL_BREAK_EVENT signals
5. **Config Persistence**: Placeholder-based configuration that works across environments

2.1.2 Architecture

MicroserviceBase is organized into distinct layers following hexagonal architecture:

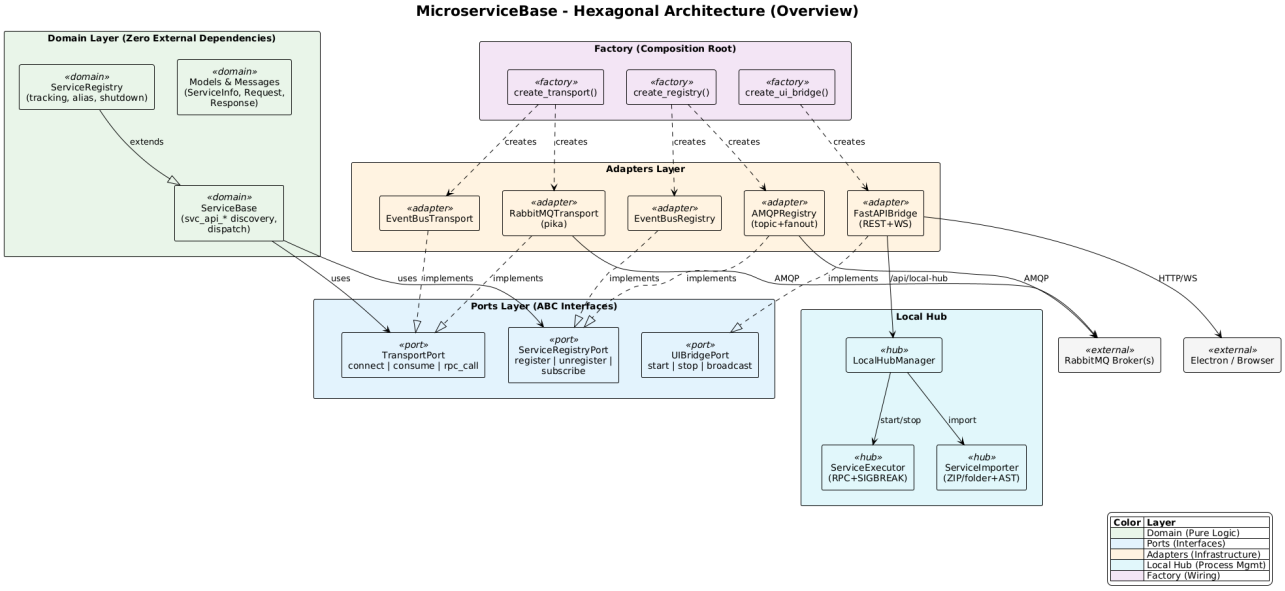


Figure 2.2: Hexagonal Architecture Overview

The detailed architecture with method signatures and implementation specifics:

Layer Responsibilities

Domain Layer Core business logic: `ServiceBase` (base class for microservices), `ServiceRegistry` (service registration and discovery), and `Factory` (component creation).

Ports Layer Abstract interfaces defining contracts: `TransportPort` (message passing), `RegistryPort` (service registration), `UIBridgePort` (GUI communication).

Adapters Layer Concrete implementations:

- `RabbitMQ Transport` – RPC calls and message consumption via pika
- `RabbitMQ Registry` – Service registration via exchange broadcasting

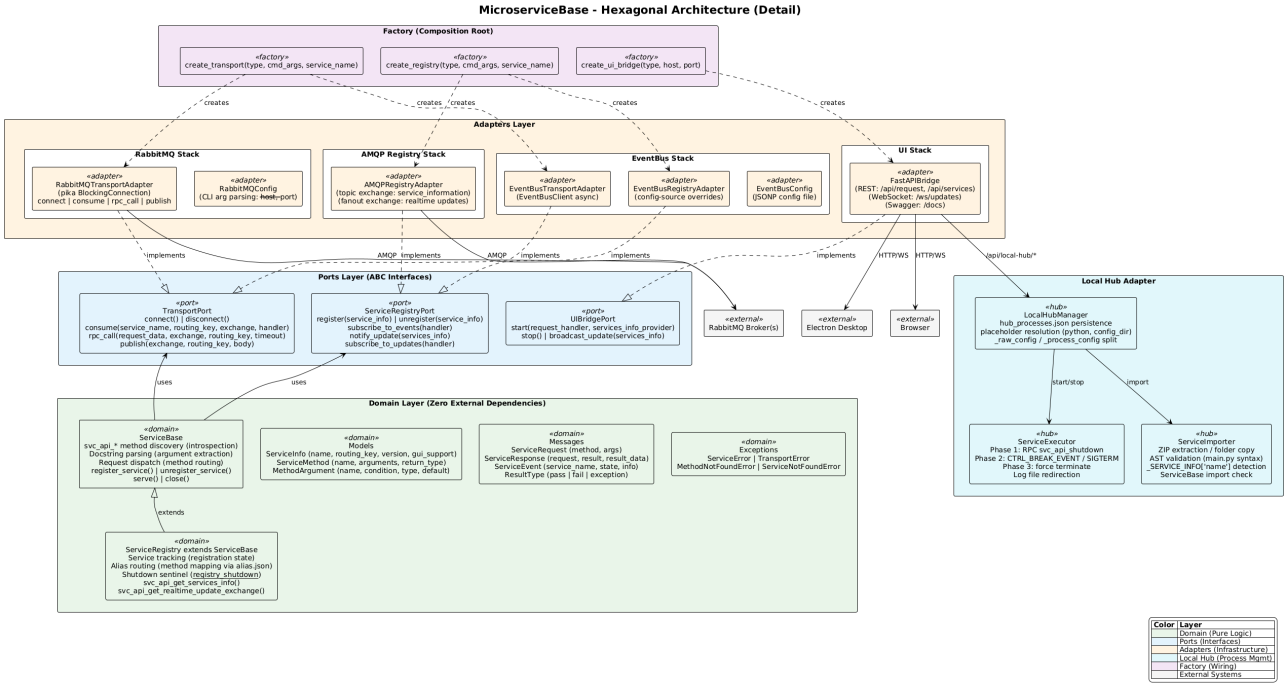


Figure 2.3: Hexagonal Architecture Detail

- `FastAPI Bridge` – REST API, WebSocket, and Local Hub endpoints
- `Local Hub Manager` – Process lifecycle management
- `Service Executor` – Process execution with platform-specific shutdown

GUI Layer MicroserviceManagerGUI with dual hosting:

- `Electron` – Desktop wrapper with contextBridge preload
- `Web` – Pure browser-compatible HTML/CSS/JS
- `Service Plugins` – Dynamically loaded per-service UI components

2.1.3 Components

ServiceBase

The base class for all microservices provides:

- API method discovery via `svc_api_` prefix convention
- Automatic service registration with the registry
- Graceful shutdown with `unregister_service()` on exit
- Service info exposure for discovery via `_SERVICE_INFO` dict
- Docstring-based API documentation generation

```
from MicroserviceBase.domain.service_base import ServiceBase
from MicroserviceBase.factory import create_transport, create_registry

class MyService(ServiceBase):
    _SERVICE_INFO = {
        'name': 'MyService',
        'routing_key': 'service.myservice',
        'version': '1.0.0',
        'gui_support': False,
        'methods': [], 'methods_info': {},
    }
```

```
    def svc_api_my_method(self, arg1, arg2):
        """My API method."""
        return some_result

transport = create_transport('rabbitmq',
    cmd_args=['--host', 'localhost', '--port', '5672'],
    service_name='MyService')
registry = create_registry('rabbitmq',
    cmd_args=['--host', 'localhost', '--port', '5672'],
    service_name='MyService')

service = MyService(transport=transport, registry=registry)
service.register_service()
service.serve()
```

Service Properties (_SERVICE_INFO)

Every microservice must define a `_SERVICE_INFO` class-level dictionary that describes the service to the framework. The registry, the GUI, and the service creator all rely on these properties. The following table lists every recognized key:

Property	Type	Required	Default	Description
`name`	str	yes	—	Unique service name (PascalCase recommended)
`description`	str	no	`''`	Full description, displayed in the Code Helper modal
`shortdesc`	str	no	`''`	Short description shown in the GUI sidebar
`group`	str	no	`''`	Category group for sidebar organization (e.g. `examples`)
`tag`	str	no	`''`	Optional version tag
`version`	str	no	`'1.0.0'`	Semantic version string
`routing_key`	str	no	`''`	RabbitMQ routing key for RPC requests
`gui_support`	bool	no	`False`	Whether the service ships a GUI plugin (`GUIs/` folder)
`downloadable`	bool	no	`False`	Whether the service exposes `svc_api_get_service_files` for remote download
`methods`	list	auto	`[]`	*Auto-populated* by `ServiceBase.__init__` from `svc_api_*` methods
`methods_info`	dict	auto	{}	*Auto-populated* from method docstrings

Custom properties (e.g. `sample_path`) may be added freely—the framework ignores unknown keys but they are broadcast with the service info, so the GUI or other clients can read them.

Example

```
_SERVICE_INFO = {
    'name': 'ServiceCleware',
    'description': 'Service to control Cleware switch box devices.',
    'shortdesc': 'Cleware Controllers',
    'group': 'Switch Boxes',
    'tag': '',
    'version': '1.0.0',
    'routing_key': 'ServiceClewareKey',
```

```
    'gui_support': True,
    'downloadable': False,
    'methods': [],       # auto-populated at __init__
    'methods_info': {},  # auto-populated from docstrings
}
```

Generated Service Template

The **Service Creator** wizard (or the `POST /api/scaffold/generate` REST endpoint) produces a ready-to-run service package with the following file structure:

```
<ServiceName>/
|-- <service_name>.py     # Service class with _SERVICE_INFO and svc_api_* methods
|-- main.py               # Entry point: creates transport, registry, registers, serves
|-- __main__.py           # Enables: python -m <ServiceName>
|-- config.jsonp          # (EventBus only) transport configuration
|-- GUIs/                 # (if gui_support is enabled)
|   |-- service.html      # Bootstrap card template (or custom HTML)
|   |-- service.js        # IIFE with load/unload functions
```

Service class (`<service_name>.py`) contains the `_SERVICE_INFO` dict with all metadata, and one `svc_api_*` method stub per API method defined in the wizard. Each method stub includes a full docstring with `**Arguments:**` and `**Returns:**` sections following the project convention.

Entry point (`main.py`) configures logging, creates the transport and registry via the factory, instantiates the service, calls `register_service()`, and enters `serve()`. Graceful shutdown is handled in a `try/except/finally` block that calls `unregister_service()` and `close()`.

Package entry point (`__main__.py`) allows the service to be started with `python -m <ServiceName>` from its parent directory, which is how the Local Hub launches imported services.

GUI plugin (`GUIs/`) is only generated when the **Generate GUI template** toggle is enabled in step 3 of the wizard. The HTML file provides a Bootstrap card layout and the JS file exposes `load<ServiceName>()` / `unload<ServiceName>()` functions using the `window.MicroserviceManager` namespace.

Service Registry

The registry manages dynamic service discovery:

- Services register on startup with their method list and metadata
- Registry broadcasts updates to all subscribers via RabbitMQ exchange
- Shutdown notification via `__registry_shutdown__` sentinel
- GUI receives real-time updates via WebSocket

FastAPI Bridge

The bridge connects the GUI to microservices:

POST /api/request RPC call forwarding to services

GET /api/services Registered service list

WS /ws/updates Real-time service update stream

/api/local-hub/* Local process hub management endpoints

GET /docs Auto-generated Swagger API documentation

Local Hub Manager

The Local Hub provides process lifecycle management:

- Start, stop, and monitor local service processes
- Process configuration via `hub_processes.json` with placeholder support
- Service import from folders or zip archives
- Service executor with graceful shutdown using `CTRL_BREAK_EVENT` on Windows
- Log file management per process

2.1.4 Communication Patterns

RPC via RabbitMQ

Services communicate using RPC (Remote Procedure Call) pattern through RabbitMQ. The following diagram shows the complete communication flow between a service, the Service Registry, and a client, including all exchanges and routing keys used:

```
from MicroserviceBase.domain.service_base import ServiceBase

# Client sends request
request_data = ServiceBase.create_request_data('svc_api_add', [1, 2])
response = transport.rpc_call(
    request_data,
    exchange_name='services_request',
    routing_key='service.calculator'
)
# response = {"request": "svc_api_add", "result": "pass",
#               "result_data": 3}
```

Service Registration

Services register themselves via the registry exchange. The `ServiceBase` class handles this automatically using the `_SERVICE_INFO` dict:

```
# ServiceBase auto-discovers svc_api_* methods and builds
# the service info dict, then publishes it:
service.register_service()

# On shutdown, unregister to notify the registry:
service.unregister_service()
```

Multi-Broker Architecture

The GUI supports connecting to multiple RabbitMQ brokers simultaneously:

- `MM.connections` map tracks per-broker state
- `serviceToBroker` reverse lookup for request routing
- Session persistence via `sessionStorage`
- Progressive disclosure: broker sections shown only when multiple brokers connected

2.1.5 GUI Architecture

Dual Hosting

The MicroserviceManagerGUI runs in two modes:

Electron Mode Desktop application with native process management. Uses `contextBridge` preload for secure Node.js access. Can spawn and manage the FastAPI bridge process directly.

Browser Mode Access via `http://localhost:1112` when the bridge is running. Pure HTML/CSS/JS with no Node.js dependencies. Uses the same `window.MicroserviceManager` namespace.

Service Plugins

Each microservice can provide a custom GUI plugin:

- HTML file for UI layout
- JS file for logic (uses `const MM = window.MicroserviceManager;`)
- Plugins loaded dynamically into `web/services/`
- In packaged mode, plugins extracted to writable `%APPDATA%` directory

Standalone Installer

The GUI can be packaged as a standalone Windows application using Electron Builder:

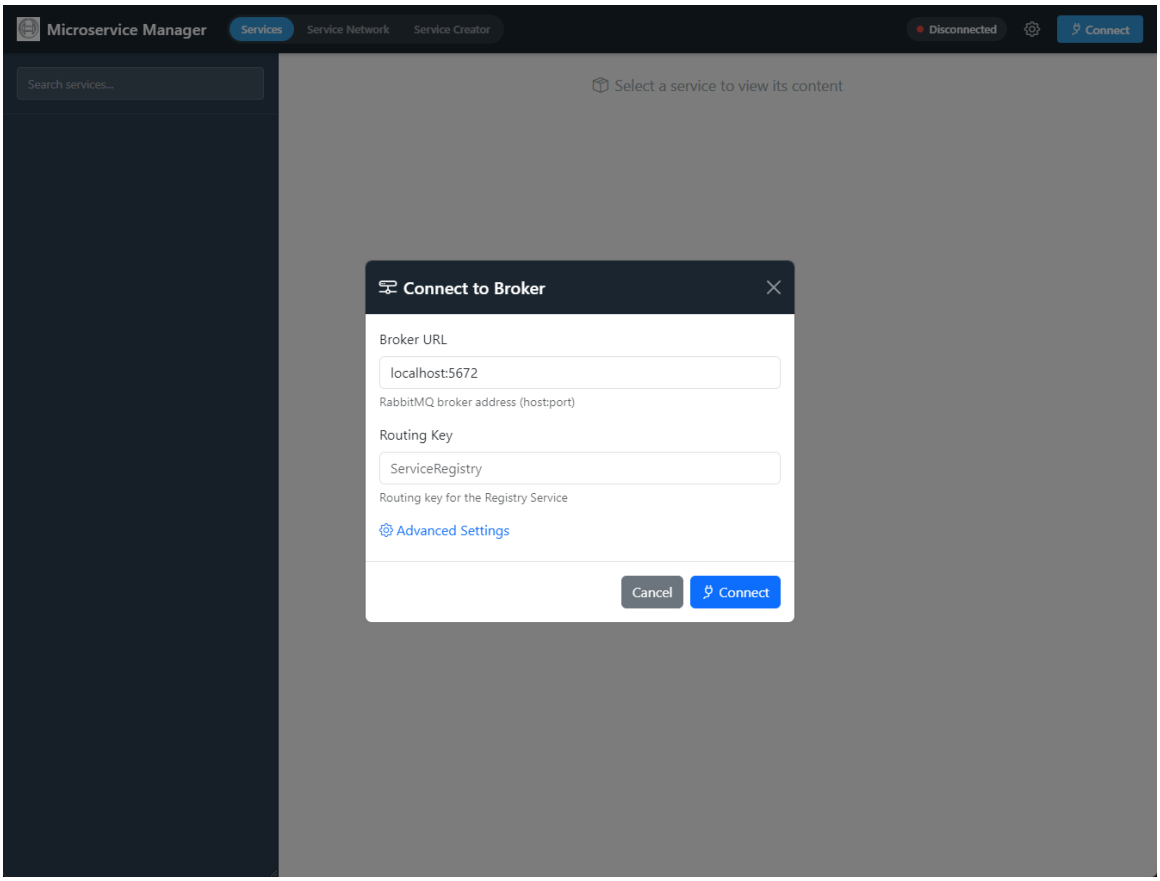
- NSIS installer for Windows (`build.bat`), Linux packages (`build.sh`)
- Read-only scripts in `resources/python/`
- Writable user data in `%APPDATA%/devatservgui/`
- First-run initialization copies templates and scripts
- Bridge process managed via PID file with detached mode

2.1.6 GUI User Guide

The Microservice Manager GUI provides three main tabs: **Services** for connecting to brokers and interacting with microservices, **Service Network** for managing the local process hub, and **Service Creator** for scaffolding new microservices.

Connecting to a Broker

To start using the GUI, click the **Connect** button in the top-right corner. The connection dialog requires two fields:



Connect to Broker Dialog

Broker URL The RabbitMQ broker address in `host:port` format (e.g., `localhost:5672`). The FastAPI bridge will connect to this broker to discover and communicate with services.

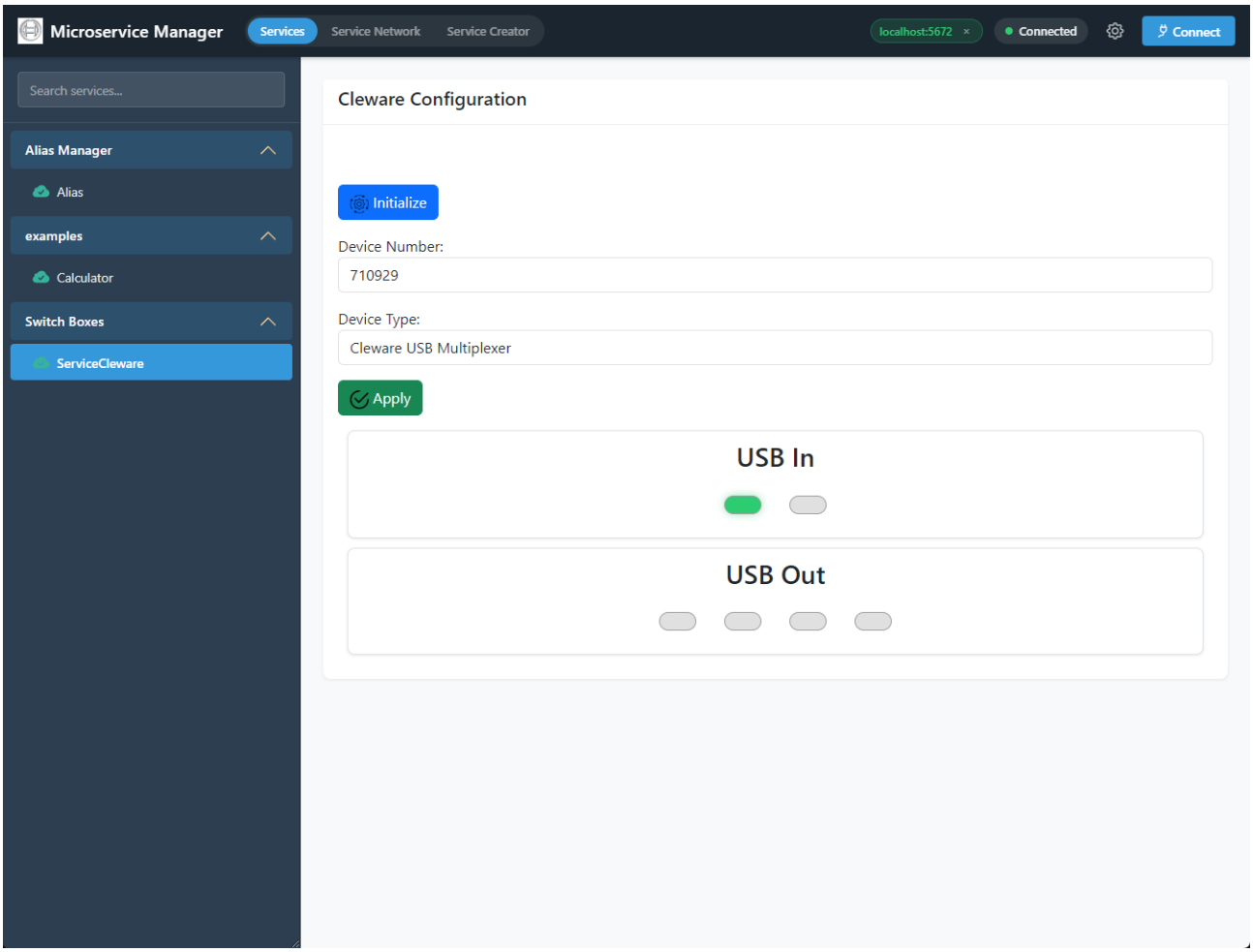
Routing Key The routing key of the Service Registry (default: `ServiceRegistry`). This tells the bridge where to find the registry service that tracks all registered microservices.

Advanced Settings Click the **Advanced Settings** link to reveal additional connection options such as exchange names and other broker-specific parameters.

After connecting, the GUI subscribes to real-time service updates via WebSocket. The connected broker address appears as a badge in the top-right corner (e.g. `localhost:5672 x`). Multiple brokers can be connected simultaneously—each broker appears as a separate section in the sidebar when more than one is active.

Service Dashboard

Once connected, the **Services** tab displays all registered microservices in the left sidebar, grouped by their service group. Clicking a service loads its details and custom GUI plugin (if available) in the main content area.

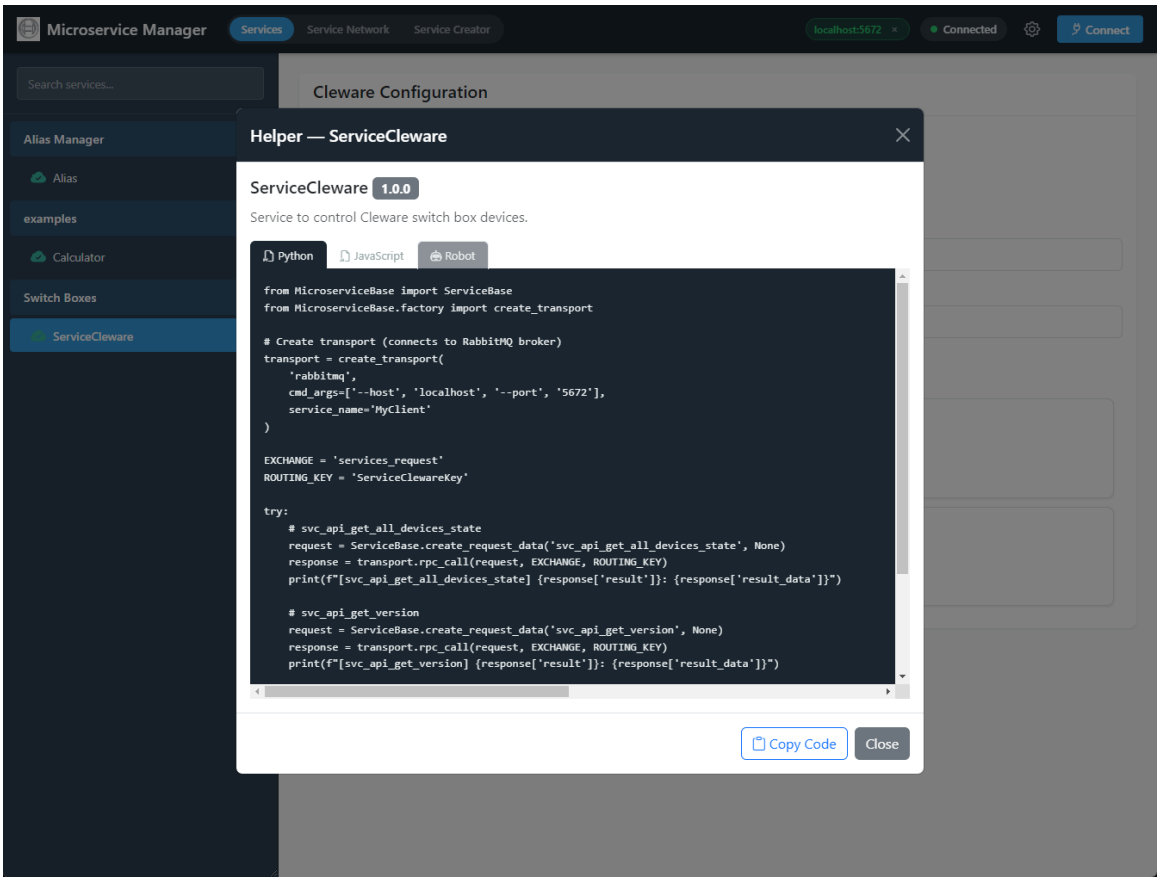


Service Dashboard with Sidebar and Service Plugin

The sidebar shows each service as an accordion item with the service name, version, and an expand arrow. Expanding a service item reveals its API methods as clickable list items. Services that provide a custom GUI plugin (indicated by `gui_support: True` in their `_SERVICE_INFO`) display their plugin interface in the main panel when selected.

Code Helper

For each service, the GUI provides auto-generated code examples that demonstrate how to call the service's API methods via RPC. Click the **Helper** button (code icon) on any service to open the code example modal.



Code Helper Modal with Python, JavaScript, and Robot Tabs

The modal header shows the service name, version badge, and description. Three language tabs are available:

Python Complete boilerplate for creating a transport connection, building a request with `ServiceBase.create_request_da` and calling the service via `rpc_call()` with the correct exchange name and routing key. Copy directly into a Python script.

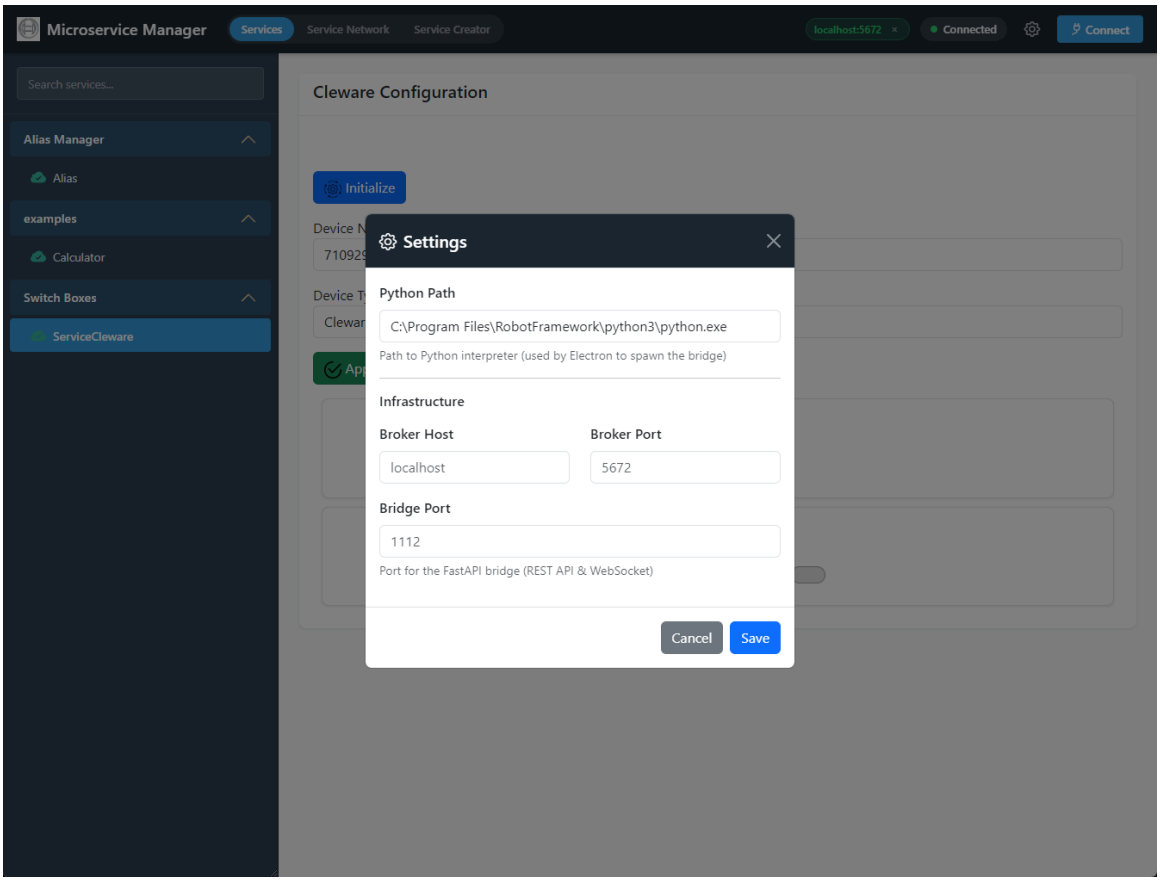
JavaScript Browser-side code using the `window.MicroserviceManager` namespace and the FastAPI bridge REST endpoint to call service methods from a web application.

Robot Robot Framework keyword examples for calling the service's API methods, ready to integrate into test automation workflows.

Click **Copy Code** to copy the currently visible tab content to the clipboard.

Settings

Click the gear icon in the top-right corner to open the **Settings** dialog.



Application Settings

Python Path The Python executable used by the Local Hub to spawn service processes. Defaults to the system Python.

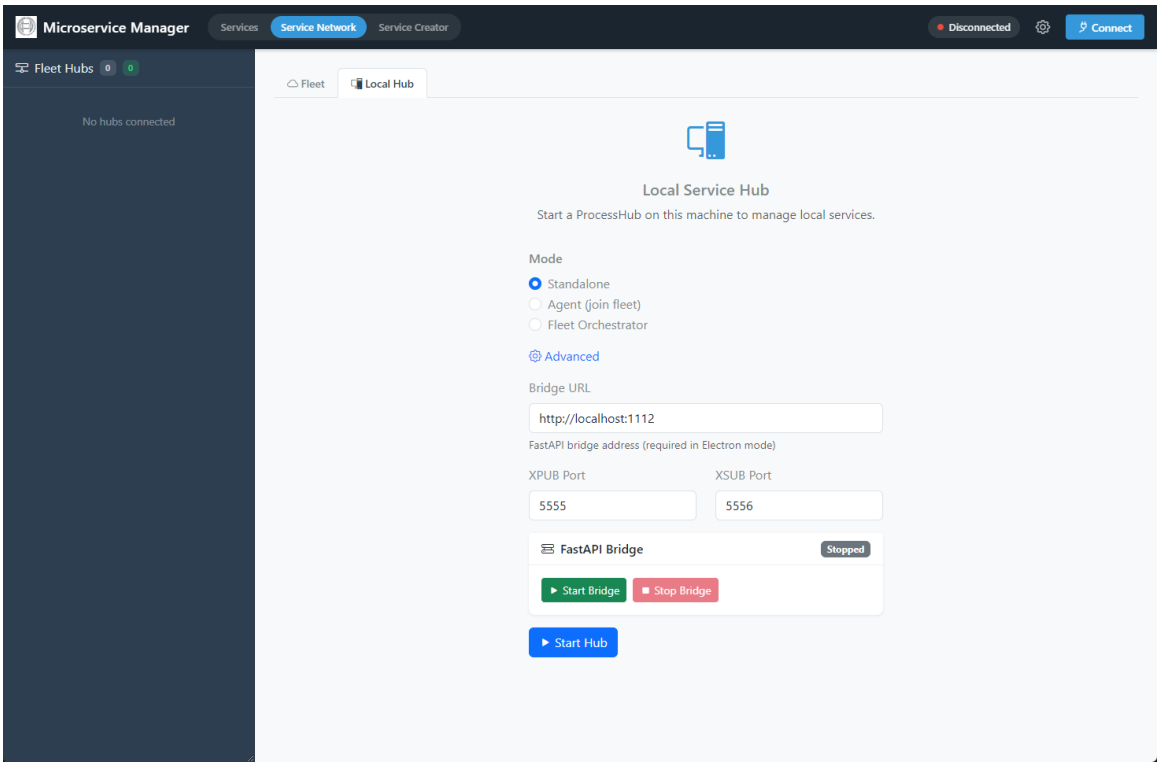
Broker Host / Port Default RabbitMQ broker connection parameters, pre-filled in the Connect dialog.

Bridge Port The port on which the FastAPI bridge listens (default: `1112`). In browser mode, access the GUI at `http://localhost:<bridge-port>`.

Settings are persisted to `settings.json` and loaded on startup. In Electron mode, the file is stored alongside the application; in packaged mode, it is stored in `%APPDATA%/DevAtServGUI/`.

Local Service Hub

The **Service Network** tab provides access to the Local Hub, which manages service processes on the local machine. Switch to the **Local Hub** sub-tab to configure and start the hub.



Local Hub Configuration and Bridge Status

The Local Hub configuration page provides the following options:

Mode Three radio buttons select the hub operating mode:

- **Standalone** — runs an independent hub on this machine (default).
- **Agent (join fleet)** — connects to an existing fleet orchestrator as a managed node.
- **Fleet Orchestrator** — starts this hub as a fleet manager that other hubs can join for centralized orchestration.

Advanced Click the **Advanced** link to reveal additional options.

Bridge URL The address of the FastAPI bridge (e.g., `http://localhost:1112`). In Electron mode, the bridge can be started directly from this page.

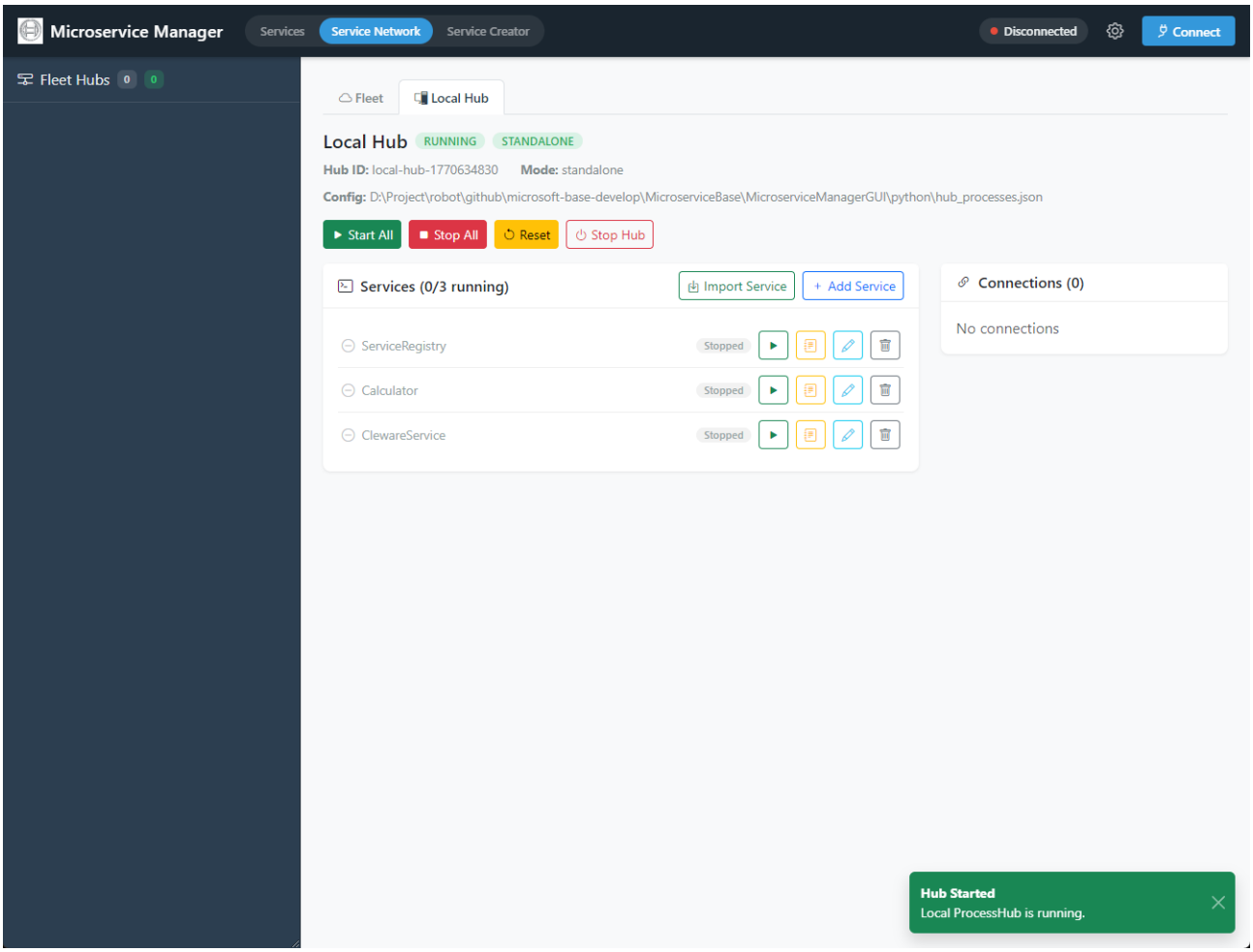
XPUB / XSUB Port Ports used by the EventBus proxy (defaults: `5555` / `5556`). Only relevant when using EventBus transport.

FastAPI Bridge A status card shows whether the bridge is *Running* or *Stopped*. Use the **Start Bridge** / **Stop Bridge** buttons to control the bridge process directly (Electron mode only).

Start Hub Click to start the Local Hub. The hub reads `hub_processes.json` and makes all configured services available for management.

Local Hub Dashboard

After starting the hub, the dashboard displays hub metadata, all configured services with their current status, and control buttons.



Local Hub Dashboard with Service Process Management

The dashboard header shows **Hub ID**, **Mode** (standalone / agent / orchestrator), and the path to the loaded **Config** file. A toast notification confirms when the hub has started successfully. The dashboard provides:

Hub Controls **Start All** and **Stop All** buttons to batch-control all services. **Reset** reloads the configuration. **Stop Hub** shuts down the hub and all running processes.

Service List A panel labelled **Services (N/M running)** shows each configured service with its name, status (*Stopped* / *Running*), and action buttons: start (▶), save config, edit configuration, and delete.

Import Service Click to import a microservice package from a folder or ZIP archive. The importer validates the package structure (checks for `main.py` with a `ServiceBase` subclass) and adds it to the hub configuration.

Add Service Manually add a new service entry to the hub configuration by specifying the script path, arguments, and process name.

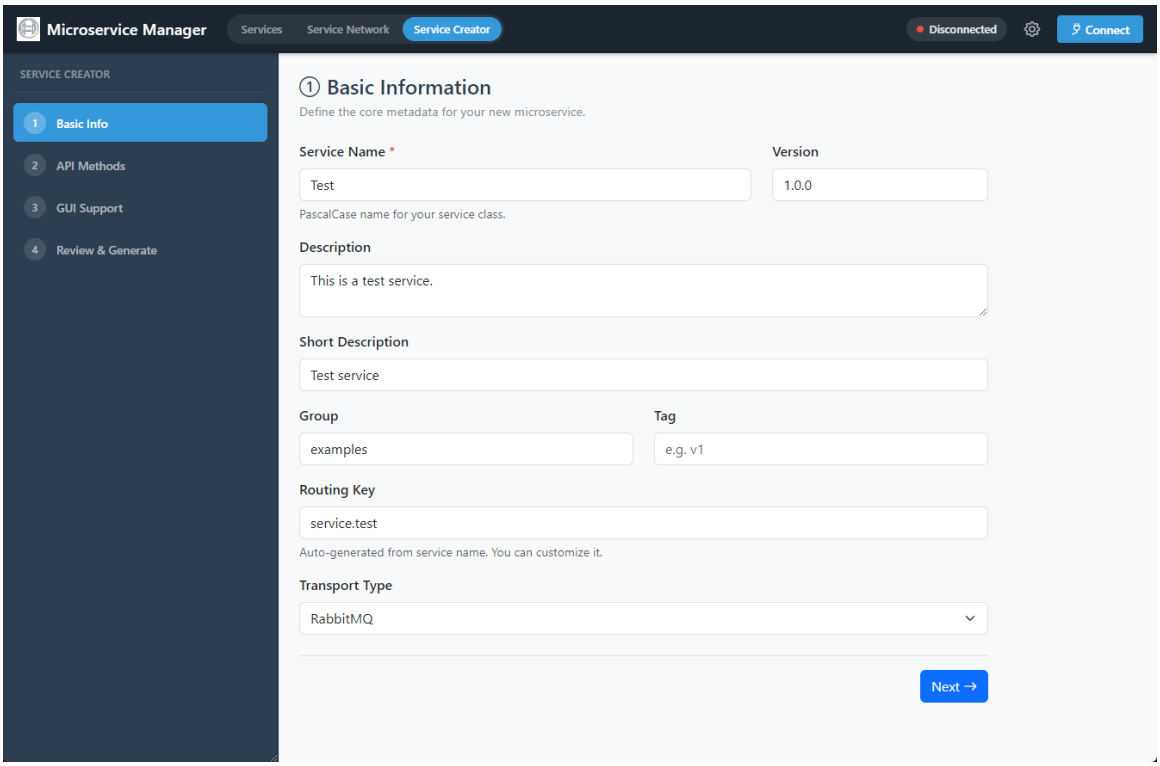
Connections A side panel shows active broker connections with the connection count badge.

Services are started as subprocess with log file redirection. The hub uses a two-phase graceful shutdown: first an RPC `svc_api_shutdown` call, then `CTRL_BREAK_EVENT` (Windows) or `SIGTERM` (Linux), and finally a force terminate if needed.

Service Creator Wizard

The **Service Creator** tab provides a step-by-step wizard for scaffolding new microservice projects. The wizard generates a complete, ready-to-run service package with all required files.

Step 1: Basic Information



Service Creator – Step 1: Basic Information

Define the core metadata for the new microservice:

Service Name PascalCase name for the service class (e.g., `Test`).

Version Semantic version string (default: `1.0.0`).

Description / Short Description Documentation strings included in the generated `_SERVICE_INFO` dict.

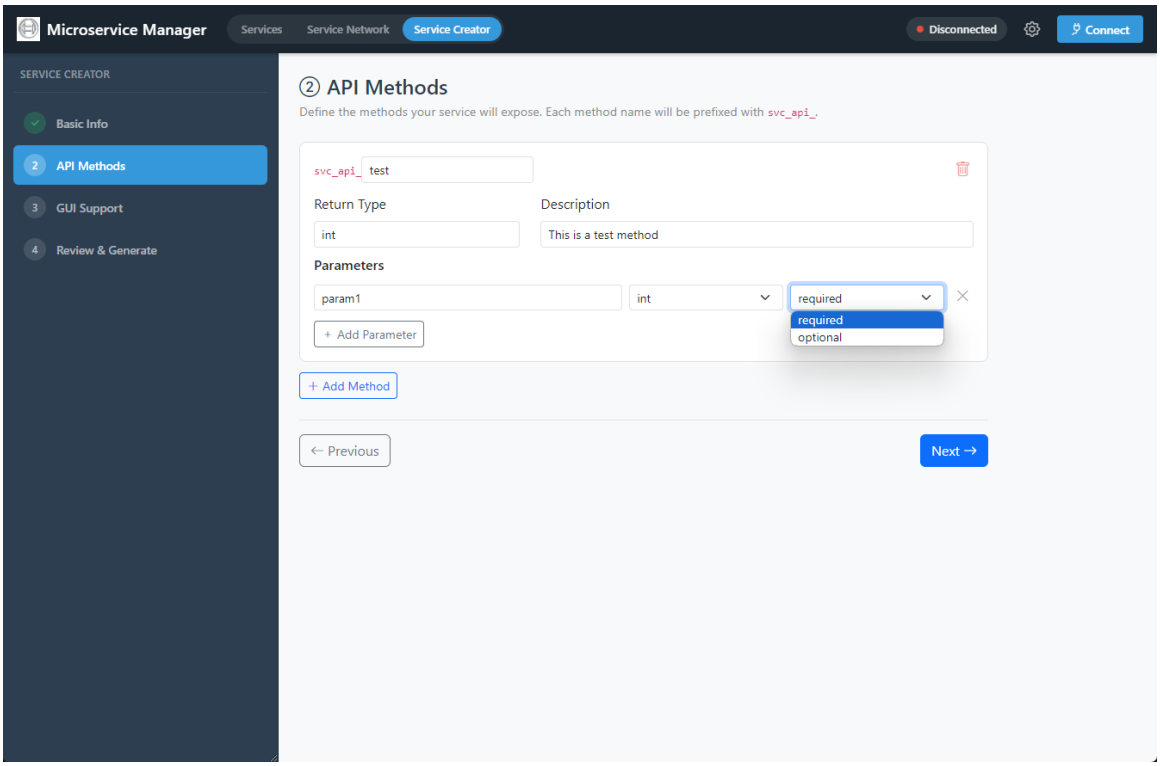
Group Service group for sidebar organization in the GUI (e.g., `examples`).

Tag Optional version tag for the service.

Routing Key Auto-generated from the service name in `service.<name>` format. Can be customized.

Transport Type Choose between `RabbitMQ` or `EventBus` transport adapter.

Step 2: API Methods

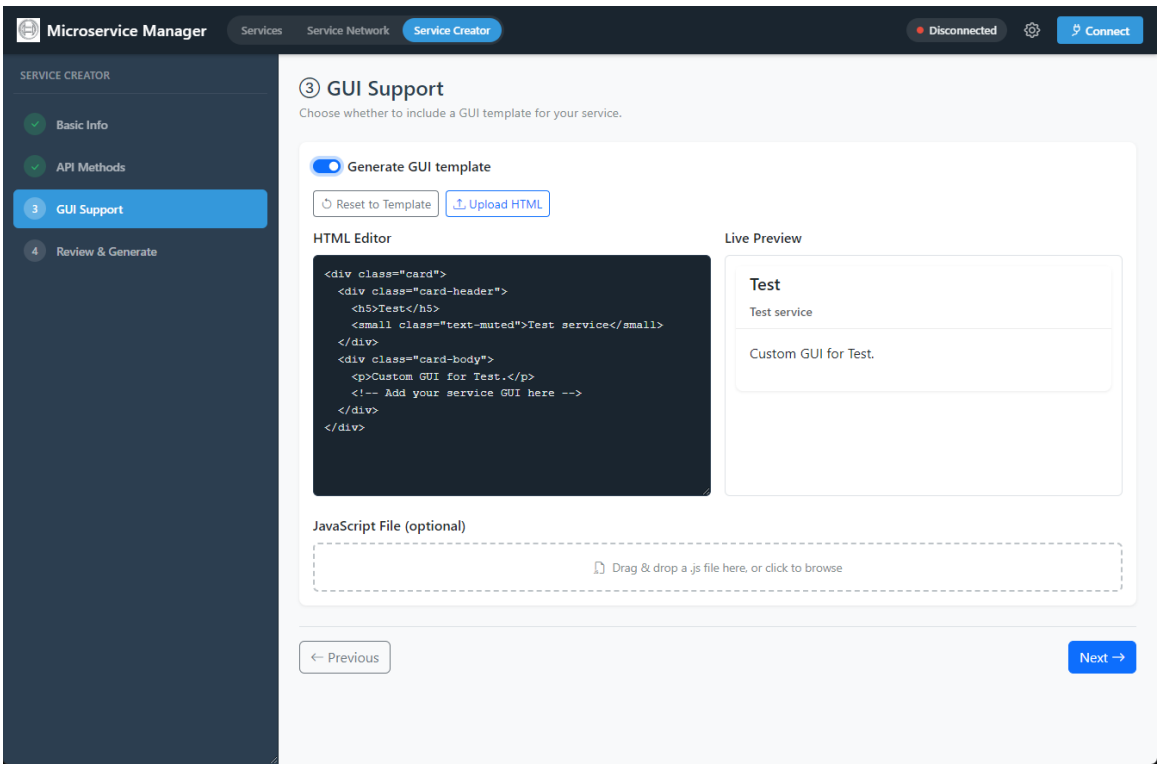


Service Creator – Step 2: API Methods

Define the methods your service will expose. Each method name is automatically prefixed with `svc_api_` following the framework convention. For each method:

- **Return Type**: The expected return type (e.g., `int`, `str`, `dict`).
- **Description**: Docstring for the method, used in API documentation generation.
- **Parameters**: Add parameters with name, type, and condition (`required` or `optional`). Click **+ Add Parameter** to add more. Click **+ Add Method** to define additional API methods.

Step 3: GUI Support



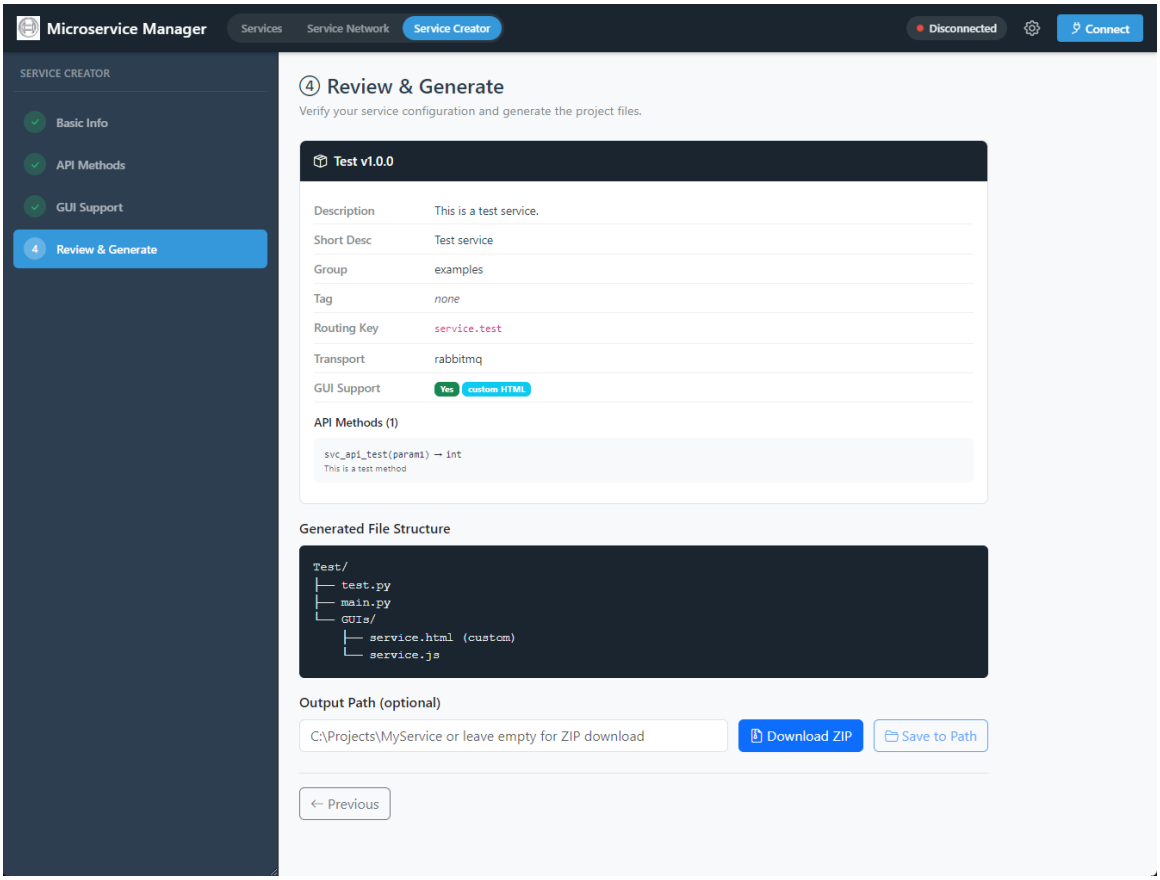
Service Creator – Step 3: GUI Support

Choose whether to include a custom GUI plugin for the service. When the **Generate GUI template** toggle is enabled:

- An **HTML Editor** provides a Bootstrap card template with the service name and description. Edit the HTML directly or click **Upload HTML** to use a custom file.
- A **Live Preview** panel shows the rendered HTML in real-time.
- An optional **JavaScript File** can be uploaded via drag-and-drop for service-specific logic (using the `window.MicroserviceManager` namespace).
- Click **Reset to Template** to restore the default HTML template.

The generated GUI plugin will be placed in a `GUIs/` folder inside the service package and loaded automatically when the service is selected in the Manager GUI.

Step 4: Review and Generate



Service Creator – Step 4: Review and Generate

Review the complete service configuration before generating:

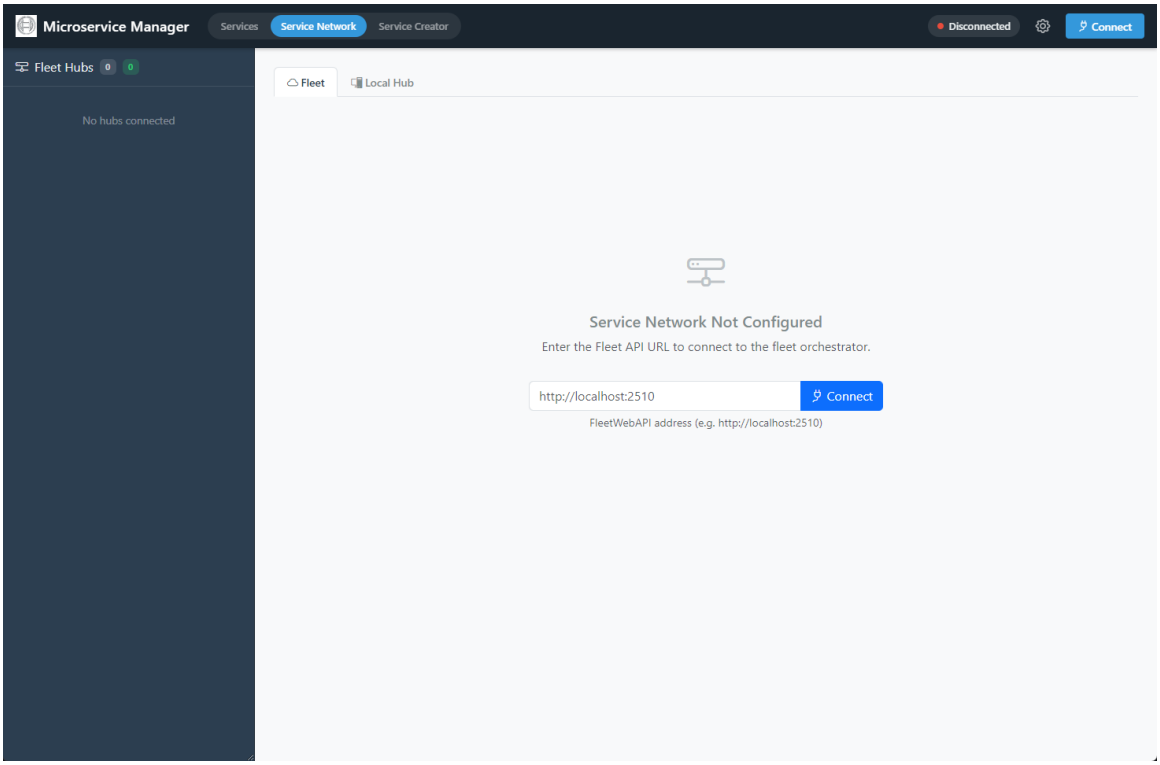
- A summary card displays all metadata, routing key, transport type, GUI support status, and API method signatures with parameter types.
- The **Generated File Structure** preview shows the directory layout that will be created (e.g., `Test/test.py`, `Test/main.py`, `Test/GUIs/service.html`).
- Specify an **Output Path** to save directly to disk, or leave empty to download as a ZIP file.
- Click **Download ZIP** to get the package as a zip archive, or **Save to Path** to write files to the specified directory.

The generated service package is ready to run. It can be imported into the Local Hub using the **Import Service** feature, or started manually via `python -m <ServiceName>` from the package directory.

Fleet Orchestrator

The **Fleet** sub-tab in the **Service Network** page provides centralized orchestration of multiple hubs across different machines.

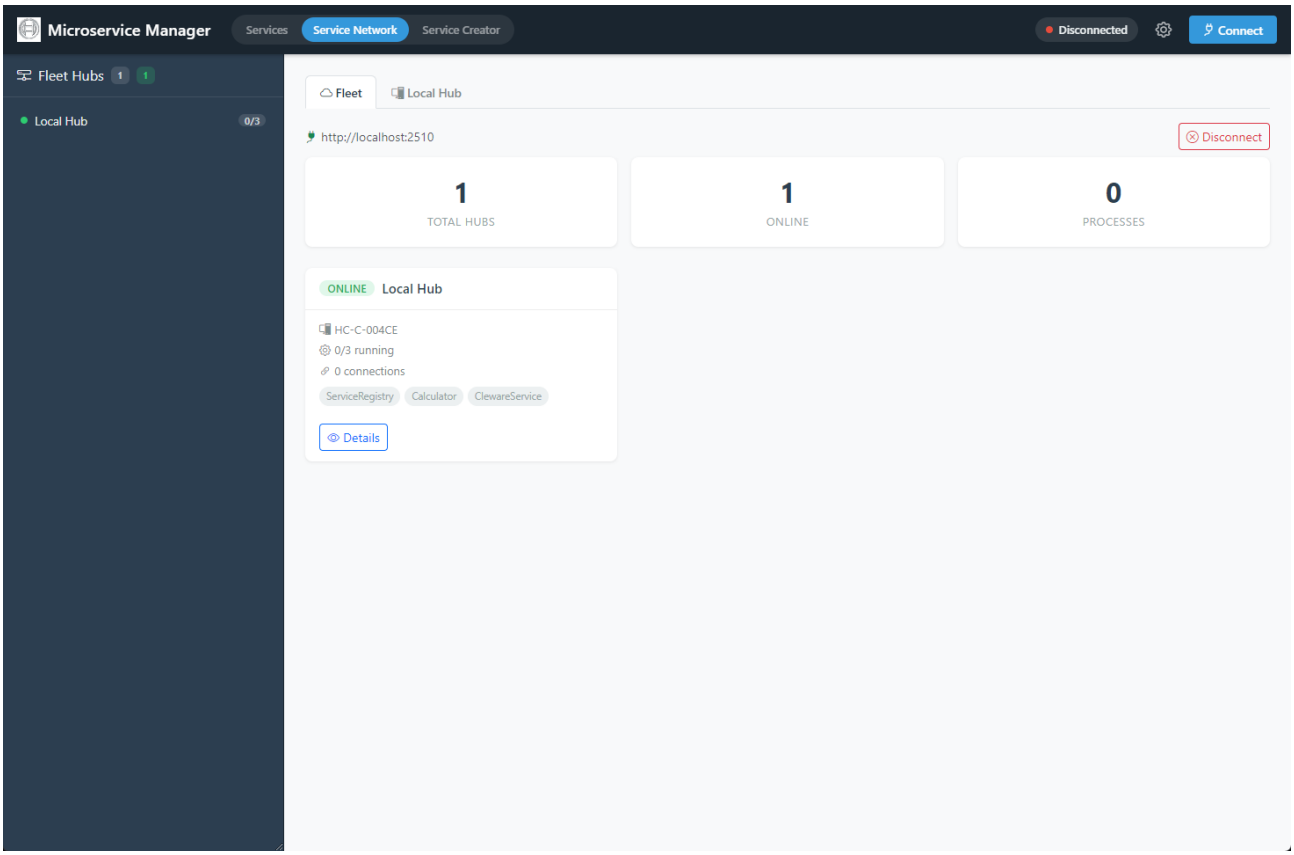
Connecting to a Fleet



Fleet Connection – Enter the Fleet API URL

When no fleet connection is configured, the page displays a **Service Network Not Configured** prompt. Enter the Fleet API URL (e.g., `http://localhost:2510`) and click **Connect** to connect to an existing fleet orchestrator. The Fleet API is a WebAPI exposed by a hub running in *Fleet Orchestrator* mode.

Fleet Dashboard



Fleet Dashboard – Overview of All Connected Hubs

Once connected, the Fleet Dashboard shows:

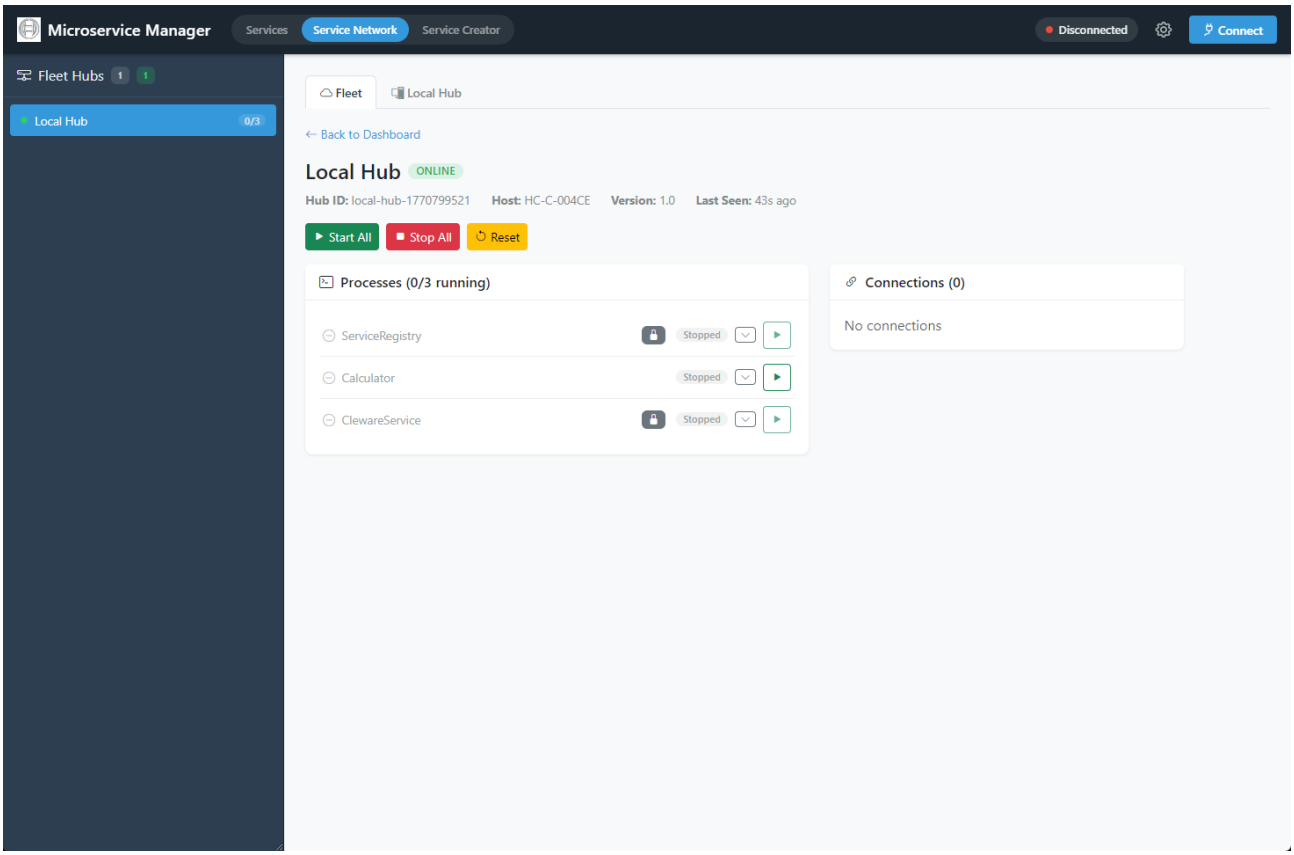
Summary Cards Three metric cards at the top display **Total Hubs**, **Online** hubs, and total **Processes** count across all hubs.

Hub Cards Each connected hub appears as a card showing its name, hostname, running process count, connection count, and the names of configured services as badges. Click **Details** to drill into a specific hub.

Sidebar The left sidebar lists all fleet hubs with their online status indicator and process count badge.

Disconnect Click **Disconnect** in the top-right corner to leave the fleet.

Fleet Hub Details



Fleet Hub Details – Remote Process Management

Clicking **Details** on a hub card opens its detail view, which mirrors the Local Hub Dashboard but operates on the remote hub:

- **Hub metadata**: Hub ID, hostname, version, and last-seen timestamp.
- **Start All / Stop All / Reset**: Batch-control all processes on the remote hub.
- **Processes**: Lists each process with status, a lock icon for fleet-protected processes (`fleet_enabled: false`), an expand button, and a start/stop button.
- **Connections**: Shows broker connections active on the remote hub.
- Click **Back to Dashboard** to return to the fleet overview.

2.1.7 Process Configuration

Processes are configured via `hub_processes.json`:

```
{
    "ServiceRegistry": {
        "script": "${config_dir}/start_registry.py",
        "args": ["--config", "${config_dir}/config.json"],
        "process_name": "ServiceRegistry",
        "wait_time": 2.0
    },
    "Calculator": {
        "script": "${python}",
        "args": ["path/to/calculator.py"],
        "process_name": "Calculator",
        "wait_time": 1.0,
        "fleet_enabled": true
    },
    "ClewareService": {
        "script": "${python}",
```

```
        "args": ["-m", "ClewareService"],
        "cwd": "${config_dir}/services",
        "process_name": "ClewareService",
        "wait_time": 1.0,
        "service_name": "ServiceCleware"
    }
}
```

Process Properties

Each entry in `hub_processes.json` supports the following properties:

script Path to the executable or Python script. Supports `${python}` and `${config_dir}` placeholders.

args List of command-line arguments passed to the script.

process_name Display name used in the dashboard and logs.

wait_time Seconds to wait after spawning before checking if the process is still alive (default: `2.0`).

cwd *(optional)* Working directory for the process. Required when using `-m` module execution (e.g., `python -m ClewareService`).

env *(optional)* Dictionary of extra environment variables to set for the subprocess.

service_name *(optional)* The RabbitMQ queue name the service consumes on. Used by the executor to send `svc_api_shutdown` RPC. Falls back to the entry key if omitted.

fleet_enabled *(optional, default `false`)* When `true`, the process can be started and stopped remotely via Fleet orchestrator commands. Processes without this flag are protected from remote control (shown with a lock icon in the Fleet Hub detail view).

Placeholders

${python} Resolves to `sys.executable` at runtime

${config_dir} Resolves to the directory containing `hub_processes.json`

Placeholders are preserved in `_raw_config` during save operations, allowing the same configuration file to work across different environments.

2.1.8 Windows Process Lifecycle

On Windows, process shutdown requires special handling:

- `proc.terminate()` calls `TerminateProcess` which allows no cleanup
- Service Executor uses `CTRL_BREAK_EVENT` for graceful shutdown
- `signal.SIGBREAK` handler converts to `KeyboardInterrupt` for Python cleanup
- Pika's `start_consuming()` replaced with `process_data_events(time_limit=1)` loop for interruptible consumption
- `subprocess.PIPE` blocking prevented by using `FileHandler` instead of `StreamHandler` for logging in subprocess mode

2.1.9 Installation

From Source

```
git clone https://github.com/test-fullautomation/python-microservice-base.git
cd python-microservice-base
pip install .

# Development mode
pip install -e .
```

GUI Setup

```
cd MicroserviceBase/MicroserviceManagerGUI
npm install
npm start
```

2.1.10 Quick Start

Start Service Registry

```
python MicroserviceBase/MicroserviceManagerGUI/python/start_registry.py \
    --host localhost --port 5672
```

Start FastAPI Bridge

```
python MicroserviceBase/MicroserviceManagerGUI/python/start_bridge.py \
    --host localhost --port 5672 --bridge-port 1112
```

Create a Service

```
import sys
from MicroserviceBase.domain.service_base import ServiceBase
from MicroserviceBase.factory import create_transport, create_registry

class CalculatorService(ServiceBase):
    _SERVICE_INFO = {
        'name': 'Calculator',
        'description': 'A simple calculator service.',
        'version': '1.0.0',
        'routing_key': 'service.calculator',
        'gui_support': False,
        'methods': [], 'methods_info': {},
    }

    def svc_api_add(self, a, b):
        """Add two numbers."""
        return int(a) + int(b)

transport = create_transport('rabbitmq',
    cmd_args=sys.argv[1:], service_name='Calculator')
registry = create_registry('rabbitmq',
    cmd_args=sys.argv[1:], service_name='Calculator')

service = CalculatorService(transport=transport, registry=registry)
service.register_service()
service.serve()
```

2.1.11 Diagrams Reference

The following PlantUML diagrams are available in the `docs/diagrams/` folder:

overview.puml System overview showing services, broker, registry, bridge, and GUI

architecture.puml Hexagonal architecture with domain, ports, and adapters

component.puml Component diagram with package dependencies

class_domain.puml Class diagram for domain layer

class_ports.puml Class diagram for port interfaces

class_adapters.puml Class diagram for adapter implementations

gui_architecture.puml GUI dual-host architecture (Electron + browser)

sequence_rpc.puml RPC call sequence from GUI to service

sequence_registration.puml Service registration flow

sequence_alias.puml Service alias resolution sequence

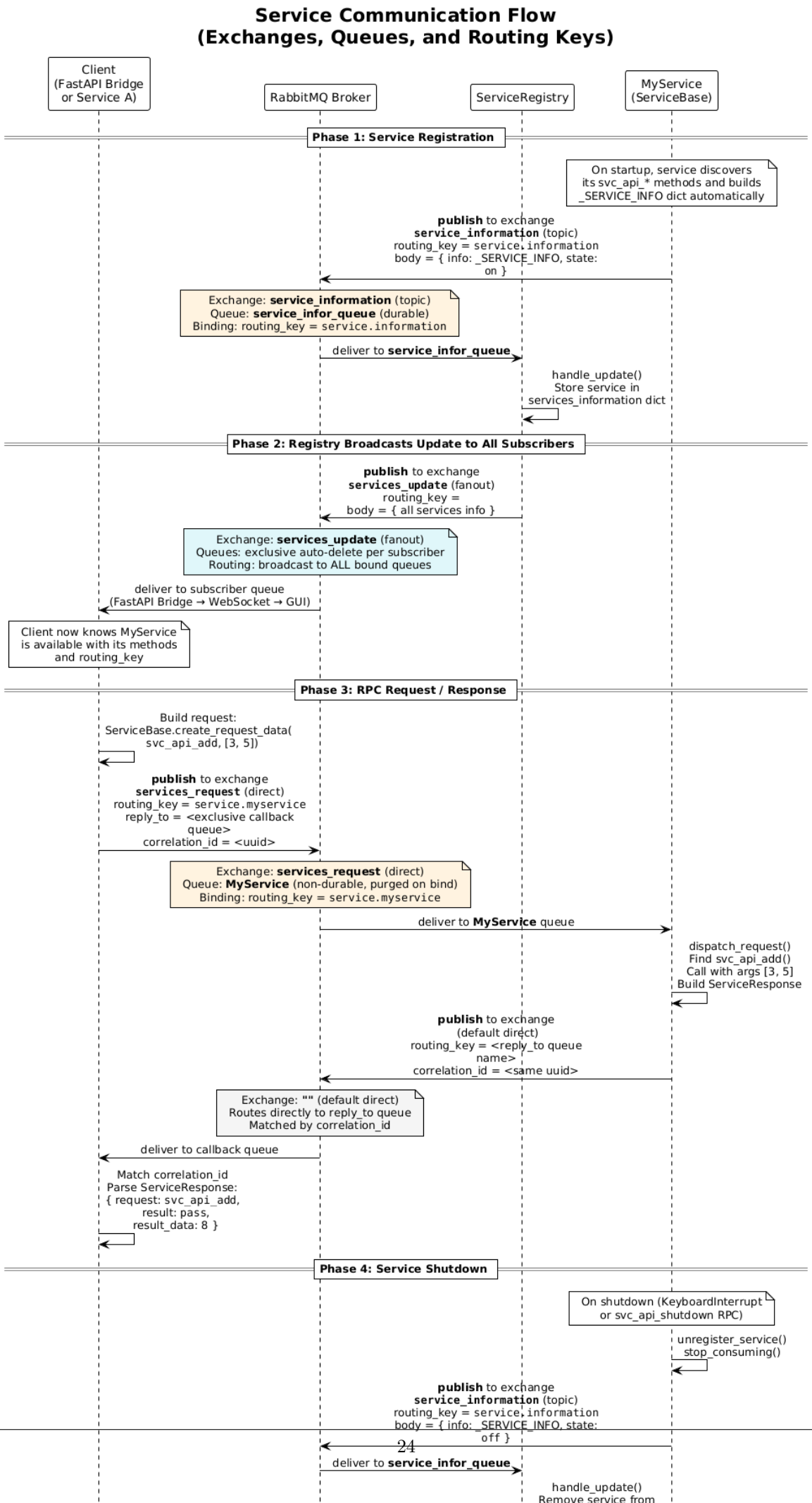
sequence_shutdown.puml Two-phase graceful shutdown sequence

sequence_service_import.puml Service import flow (GUI to hub)

state_process_lifecycle.puml Process state machine

component_local_hub.puml Local Hub internal architecture (LocalHubManager, ServiceExecutor, ProcessHub, config management)

component_fleet.puml Fleet Orchestrator multi-hub architecture (Orchestrator, Agents, FleetWebAPI, EventBus, permission guard)


Service Communication Flow
(Exchanges, Queues, and Routing Keys)
Client (FastAPI Bridge or Service A)
RabbitMQ Broker
ServiceRegistry
MyService (ServiceBase)
Phase 1: Service Registration
On startup, service discovers its svc_api_* methods and builds _SERVICE_INFO dict automatically
publish to exchange service_information (topic) routing_key = service.information body = { info: _SERVICE_INFO, state: on }
Exchange: service_information (topic) Queue: service_infor_queue (durable) Binding: routing_key = service.information
deliver to service_infor_queue
handle_update() Store service in services_information dict
Phase 2: Registry Broadcasts Update to All Subscribers
publish to exchange services_update (fanout) routing_key = body = { all services info }
Exchange: services_update (fanout) Queues: exclusive auto-delete per subscriber Routing: broadcast to ALL bound queues
deliver to subscriber queue (FastAPI Bridge → WebSocket → GUI)
Client now knows MyService is available with its methods and routing_key
Phase 3: RPC Request / Response
Build request: ServiceBase.create_request_data(svc_api_add, [3, 5])
publish to exchange services_request (direct) routing_key = service.myservice reply_to = <exclusive callback queue> correlation_id = <uuid>
Exchange: services_request (direct) Queue: MyService (non-durable, purged on bind) Binding: routing_key = service.myservice
deliver to MyService queue
dispatch_request() Find svc_api_add() Call with args [3, 5] Build ServiceResponse
publish to exchange (default direct) routing_key = <reply_to queue name> correlation_id = <same uuid>
Exchange: "" (default direct) Routes directly to reply_to queue Matched by correlation_id
deliver to callback queue
Match correlation_id Parse ServiceResponse: { request: svc_api_add, result: pass, result_data: 8 }
Phase 4: Service Shutdown
On shutdown (KeyboardInterrupt or svc_api_shutdown RPC)
unregister_service() stop_consuming()
publish to exchange service_information (topic) routing_key = service.information body = { info: _SERVICE_INFO, state: off }
deliver to service_infor_queue
handle_update() Remove service from

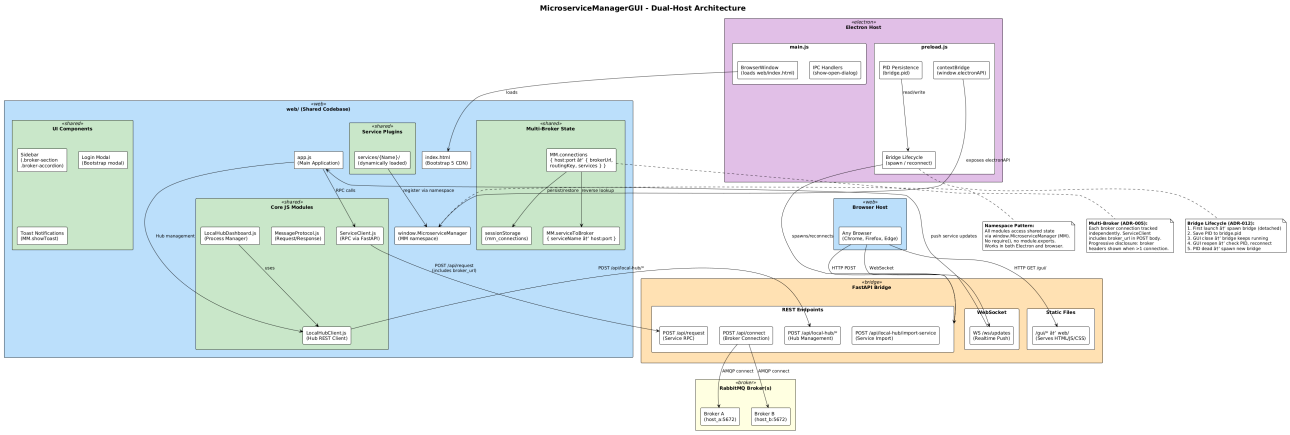


Figure 2.5: GUI Dual-Host Architecture (Electron + Browser)

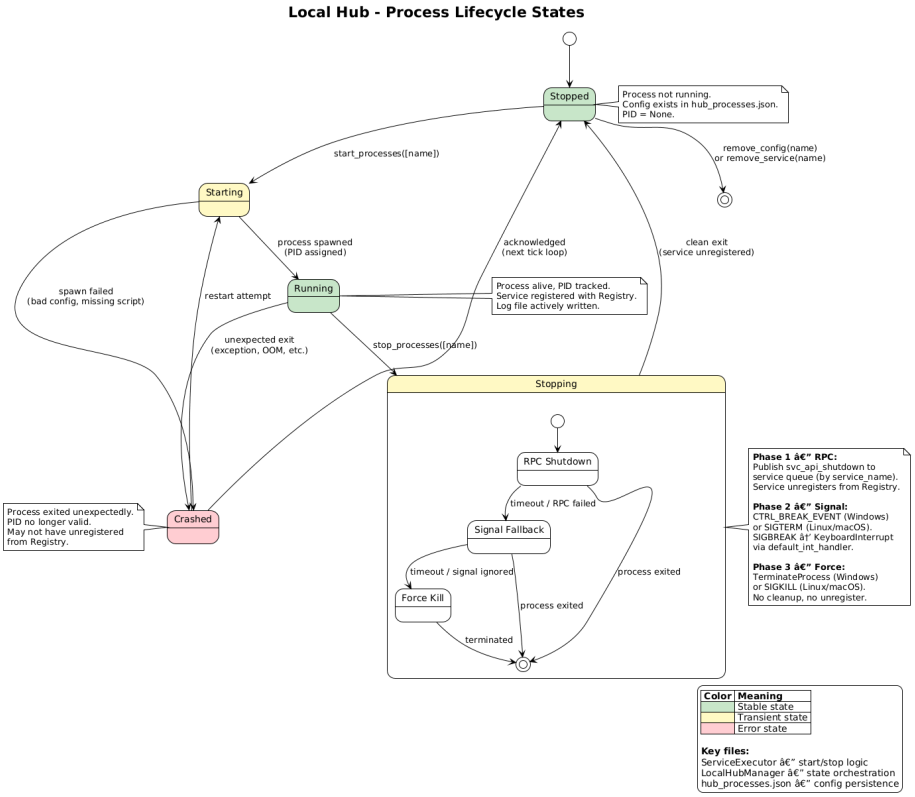


Figure 2.6: Local Hub Process Lifecycle States

Chapter 3

01_basic_service.py

Basic Microservice Example.

This example demonstrates how to: 1. Define a service by extending ServiceBase 2. Expose API methods with the svc_api prefix 3. Write docstrings that generate API documentation automatically 4. Create a transport and registry via the factory 5. Register the service and start serving requests

The service will: - Connect to RabbitMQ - Register itself with the Service Registry - Start consuming RPC requests on its routing key - Respond to method calls with structured ServiceResponse messages

3.1 Function: main

Run the Calculator service.

3.2 Class: CalculatorService

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.01_basic_ser
↪  import CalculatorService
```

A simple calculator service.

Demonstrates the svc_api convention and docstring-based API discovery.

3.2.1 Method: svc_api_add

Add two numbers.

Arguments:

- `a`
  / *Condition*: required / *Type*: int /
  First number.
- `b`
  / *Condition*: required / *Type*: int /
  Second number.

Returns:

/ *Type*: int /
Sum of a and b.

3.2.2 Method: svc_api_subtract

Subtract b from a.

Arguments:

- `a`
  / *Condition*: required / *Type*: int /
  First number.
- `b`
  / *Condition*: required / *Type*: int /
  Number to subtract.

Returns:

/ *Type*: int /
Difference of a and b.

3.2.3 Method: svc_api_multiply

Multiply two numbers.

Arguments:

- `a`
  / *Condition*: required / *Type*: int /
  First number.
- `b`
  / *Condition*: required / *Type*: int /
  Second number.

Returns:

/ *Type*: int /
Product of a and b.

Chapter 4

02_service_client.py

Service Client Example.

This example demonstrates how to: 1. Create a transport adapter for client-side RPC calls 2. Build a ServiceRequest with method name and arguments 3. Send an RPC call and receive a ServiceResponse 4. Parse the response to extract result data

Prerequisites:

- RabbitMQ server running
- Calculator service (01_basic_service.py) running

4.1 Function: main

Run the service client example.

Chapter 5

03_service_registry.py

Service Registry Example.

This example demonstrates how to: 1. Create and run the ServiceRegistry (central discovery hub) 2. Configure the realtime update exchange for UI clients 3. Handle service registration and unregistration events 4. Broadcast service list updates to all connected UI clients

The Service Registry: - Listens on the 'service_information' topic exchange for register/unregister events - Maintains a dict of all currently registered services - Broadcasts the full service list when changes occur (via fanout exchange) - Exposes its own API methods for querying service information - Supports alias routing via alias.json configuration

5.1 Function: main

Run the Service Registry.

Chapter 6

04_eventbus_transport.py

EventBus Transport Example.

This example demonstrates how to: 1. Use EventBus transport instead of direct RabbitMQ (pika) 2. Configure transport via a JSONP config file 3. Swap transports without changing the service code 4. Use the factory to create EventBus-based adapters

Prerequisites:

- RabbitMQ server running
- EventBusClient package installed

The EventBus transport provides: - JSONP-based configuration (no CLI argument parsing) - Async message handling via aio_pika under the hood - Topic and Fanout exchange handlers - Shared configuration for derivative clients (registry, updates)

Example JSONP config file (config.jsonp): { "host": "localhost", "port": 5672, "username": "guest", "password": "guest", "exchange_name": "services_request", "exchange_handler": "TopicExchangeHandler" }

6.1 Function: create_config_file

Create a temporary JSONP config file for the EventBus transport.

In production, this file would be a persistent configuration file.

6.2 Function: main

Run the Greeter service with EventBus transport.

6.3 Class: GreeterService

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.04_eventbus_
↪  import GreeterService
```

A greeting service using EventBus transport.

Same service code as any other service -- only the transport wiring differs.

6.3.1 Method: svc_api_hello

Greet someone by name.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Name of the person to greet.

Returns:

/ *Type*: str /
Greeting message.

6.3.2 Method: svc_api_goodbye

Say goodbye to someone.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Name of the person.

Returns:

/ *Type*: str /
Farewell message.

Chapter 7

05_alias_routing.py

Alias Routing Example.

This example demonstrates how to: 1. Configure method aliases in the Service Registry 2. Call services using user-friendly alias names 3. Use argument templates with ${input} substitution 4. Route requests transparently through the Registry

Alias routing allows: - User-friendly names for complex service methods - Pre-configured arguments (only ${input} needs to be provided) - Transparent forwarding (caller doesn't need to know the target service) - Centralized routing configuration

Example alias configuration: { "double": { "Service name": "Calculator", "Method name": "svc_api_multiply", "Arguments": "${input},2"
}, "add_ten": { "Service name": "Calculator", "Method name": "svc_api_add", "Arguments": "${input},10" }
}

Prerequisites:

- Calculator service (01_basic_service.py) running
- Service Registry (03_service_registry.py) running

7.1 Function: main

Run the alias routing example.

Chapter 8

06_fastapi_bridge.py

FastAPI UI Bridge Example.

This example demonstrates how to: 1. Create a FastAPI bridge for browser-based access to microservices 2. Expose a REST endpoint for invoking service methods 3. Expose a REST endpoint for listing registered services 4. Push realtime service updates via WebSocket 5. Access auto-generated Swagger documentation

The FastAPI bridge provides: - REST API for any HTTP client (curl, browser, Python requests) - WebSocket for realtime push notifications - Swagger UI at /docs for interactive API exploration - Decoupled UI layer (any frontend can connect)

Example HTTP requests: # List all services curl http://localhost:1112/api/services

> # Call a service method curl -X POST http://localhost:1112/api/request -H "Content-Type: application/json" -d '{"method": "svc_api_add", "args": [3, 5], "exchange": "services_request", "routing_key": "service.calculator"}'

8.1 Function: main

Run the FastAPI bridge example.

Chapter 9

07_mock_fleet_api.py

Mock FleetWebAPI Server.

Simulates a fleet orchestrator with multiple hubs for GUI development and demonstration purposes. Each hub has a set of processes that can be started and stopped via the API, and hub health status changes over time.

Endpoints (matching the real FleetWebAPI contract): GET /api/fleet/status - Full fleet snapshot GET /api/fleet/hubs - Hub list GET /api/fleet/hubs/{hub_id} - Single hub detail POST /api/fleet/hubs/{hub_id}/start - Start processes POST /api/fleet/hubs/{hub_id}/stop - Stop processes POST /api/fleet/hubs/{hub_id}/reset - Reset hub POST /api/fleet/command - Raw fleet command GET / - Simple status page

9.1 Function: dashboard

9.2 Function: fleet_status

9.3 Function: fleet_hubs

9.4 Function: fleet_hub_detail

9.5 Function: fleet_hub_start

9.6 Function: fleet_hub_stop

9.7 Function: fleet_hub_reset

9.8 Function: fleet_command

9.9 Function: main

9.10 Class: ProcessActionRequest

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.07_mock_flee
↪  import ProcessActionRequest
```

9.11 Class: CommandRequest

Imported by:

```
from
 ↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.07_mock_flee
 ↪  import CommandRequest
```

Chapter 10

08_fleet_demo.py

Fleet Demo — All-in-one launcher.

Starts both the mock FleetWebAPI and a GUI server so you can demo the Service Network tab in a single command with zero infrastructure.

10.1 Function: start_mock_fleet

Start the mock FleetWebAPI in a background thread. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-08-fleet-demo-start-gui-server ==========

Start a minimal FastAPI server that serves the GUI + fleet proxy. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-08-fleet-demo-main ==========

Chapter 11

run_hub_agent.py

Hub Agent with Calculator and Cleware processes.

Starts a ProcessHub server that manages two microservice processes:

- Calculator: 01_basic_service.py (arithmetic service)
- ServiceCleware: python -m MicroserviceClewareSwitch (USB switch service)

A HubAgent sidecar connects to the FleetOrchestrator so these processes can be monitored and controlled from the Service Network tab.

11.1 Function: build_process_config

Build the process configuration for Calculator and Cleware.

Each entry tells ProcessHub how to start the process. The subprocess inherits os.environ but NOT sys.path modifications, so we must pass PYTHONPATH explicitly via the 'env' dict. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-fleet-demo-run-hub-agent-main ==========

Chapter 12

run_orchestrator.py

Fleet Orchestrator.

Starts the FleetOrchestrator that receives hub agent heartbeats and routes commands, plus the FleetWebAPI on port 2510 for the GUI's Service Network tab.

Open http://localhost:2510 for the built-in fleet dashboard, or connect from the MicroserviceManager GUI.

12.1 Function: main

Chapter 13

serve_dev.py

serve_dev.py — Dev server for Qt WASM service template.

Serves the WASM build output AND provides an HTTP→RabbitMQ bridge so the Qt WASM GUI can call the running MyQtWasmService.exe backend.

Uses MicroserviceBase.adapters.transport for RabbitMQ communication — the same transport layer used by the Python microservices.

Usage: python serve_dev.py [port] [build_dir] python serve_dev.py 8090 build/wasm

Prerequisites:

- MicroserviceBase package installed (pip install MicroserviceBase)
- RabbitMQ running on localhost:5672
- MyQtWasmService.exe running

13.1 Function: load_broker_config

Load broker config from service_config.json, fall back to defaults. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-qt-wasm-service-template-serve-dev-rpc-call ==========

Send an RPC request via MicroserviceBase transport and return the response. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-qt-wasm-service-template-serve-dev-find-build-dir ==========

Auto-detect WASM build output directory. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-qt-wasm-service-template-serve-dev-main ==========

13.2 Class: DevHandler

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.qt_wasm_serv
↪  import DevHandler
```

HTTP handler: static files + RabbitMQ bridge. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-qt-wasm-service-template-serve-dev-devhandler-do-get ----------

13.2.1 Method: do_POST

13.2.2 Method: guess_type

13.2.3 Method: log_message

Only log API calls and errors, not every static file.

Chapter 14

__main__.py

Entry point for `python -m <service_folder>` execution.

Chapter 15

main.py

Service Template.

Copy this folder and customize it to create your own microservice.

Steps: 1. Rename the folder to match your service name. 2. Update _SERVICE_INFO with your service metadata. 3. Add your own svc_api methods. 4. (Optional) Set 'gui_support': True and add GUI files under GUIs/. 5. (Optional) Set 'downloadable': True to let users download this service as a ZIP from the Service Manager GUI.

15.1 Function: main

Run the service. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-service-template-main-myservice ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.service_temp
↪  import MyService
```

A template service — replace with your own description. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-service-template-main-myservice-svc-api-hello ----------

Say hello.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  The name to greet.

Returns:

/ *Type*: str /
A greeting message.

15.1.1 Method: svc_api_echo

Echo a message back.

Arguments:

- `message`
  / *Condition*: required / *Type*: str /
  The message to echo.

Returns:

/ *Type*: str /
The same message.

Chapter 16

launcher.py

FastAPI Bridge launcher.

Starts the FastAPI Bridge (REST/WS gateway) that connects the MicroserviceManagerGUI to microservices via RabbitMQ.

16.1 Function: parse_args

16.2 Function: main

Chapter 17

ClewareAccessHelper.py

17.1 Class: ClewareAccessHelper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import ClewareAccessHelper
```

ClewareAccessHelper class acts as a proxy class in Proxy pattern and forward the request to the real helper depend on platform. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-import-modules-from-paths ----------

Import all modules from given paths.

Arguments:

- `paths`
  / *Condition*: required / *Type*: list /
  List of paths to import modules from.

17.1.1 Method: load_config

Load the user config port name for USB access. Returns: None microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-get-config ----------

Get Cleware ports' config. Returns: Dictionary of ports' setting. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-set-config ----------

Save Cleware ports' config to file. Args: config_json: data to be saved.

Returns: res: saved result.

17.1.2 Method: init_cleware

17.1.3 Method: open_cleware

17.1.4 Method: close_cleware

17.1.5 Method: get_handle

17.1.6 Method: set_value

17.1.7 Method: get_value

17.1.8 Method: set_switch

17.1.9 Method: set_switch_by_port_name

17.1.10 Method: get_switch

17.1.11 Method: get_version

17.1.12 Method: get_usb_type

17.1.13 Method: get_serial_number

17.1.14 Method: valid_ser_number

17.1.15 Method: get_hw_version

17.1.16 Method: is_ampel

17.1.17 Method: iox

17.1.18 Method: get_all_devices_state

Chapter 18

ClewareAccessHelperAbs.py

18.1 Class: ClewareAccessHelperAbs

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import ClewareAccessHelperAbs
```

ClewareAccessHelperAbs acts as a abstract class for the real Helper in Proxy Pattern. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-clewareaccesshelperabs-clewareaccesshelper init-cleware ----------

18.1.1 Method: open_cleware

18.1.2 Method: close_cleware

18.1.3 Method: set_value

18.1.4 Method: get_value

18.1.5 Method: set_switch

18.1.6 Method: get_switch

18.1.7 Method: get_handle

18.1.8 Method: get_version

18.1.9 Method: get_usb_type

18.1.10 Method: get_usb_type

18.1.11 Method: get_serial_number

18.1.12 Method: valid_ser_number

18.1.13 Method: get_hw_version

18.1.14 Method: is_ampel

18.1.15 Method: iox

Chapter 19

ClewareAccessHelperLinux.py

19.1 Class: ClewareAccessHelperLinux

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import ClewareAccessHelperLinux
```

ClewareAccessHelperLinux acts as the real object on Linux platform in Proxy Pattern

- This is the wrapper class for the functions provided by C/C++ source for Linux.
- The C/C++ source for Linux is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN
- The C/C++ source for Linux is modified by Cuong Nguyen (RBVH/ENG22) for support python to load dynamic library on Linux.

19.1.1 Method: init_cleware

19.1.2 Method: open_cleware

19.1.3 Method: close_cleware

19.1.4 Method: get_handle

19.1.5 Method: set_value

19.1.6 Method: get_value

19.1.7 Method: set_switch

19.1.8 Method: get_switch

19.1.9 Method: get_version

19.1.10 Method: get_usb_type

19.1.11 Method: get_serial_number

19.1.12 Method: valid_ser_number

19.1.13 Method: get_hw_version

19.1.14 Method: is_ampel

19.1.15 Method: iox

19.1.16 Method: get_all_sw_state

Chapter 20

ClewareAccessHelperWindows.py

20.1 Class: ClewareAccessHelperWindows

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import ClewareAccessHelperWindows
```

ClewareAccessHelperWindows acts as the real object on Windows platform in Proxy Pattern

- This is the wrapper class for the functions provided by USBaccess.dll.
- The USBaccess.dll is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN

20.1.1 Method: init_cleware

20.1.2 Method: open_cleware

20.1.3 Method: close_cleware

20.1.4 Method: get_handle

20.1.5 Method: set_value

20.1.6 Method: get_value

20.1.7 Method: set_switch

20.1.8 Method: get_switch

20.1.9 Method: get_version

20.1.10 Method: get_usb_type

20.1.11 Method: get_serial_number

20.1.12 Method: valid_ser_number

20.1.13 Method: get_hw_version

20.1.14 Method: is_ampel

20.1.15 Method: iox

Chapter 21

ServiceCleware.py

21.1 Class: ServiceCleware

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import ServiceCleware
```

Service for controlling Cleware devices.

This class extends ServiceBase to provide functionalities specific to managing and controlling Cleware devices.

21.1.1 Method: svc_api_get_all_devices_state

Retrieve the state of all Cleware devices.

Returns:

/ *Type*: dict /
A dictionary containing the states of all Cleware devices.

21.1.2 Method: svc_api_set_switch

Set state for a Cleware device's switch.

Arguments:

- `device_no`
  / *Condition*: required / *Type*: str /
  Cleware device's number.
- `switch_id`
  / *Condition*: required / *Type*: str /
  Switch number to turn on or off.
- `state`
  / *Condition*: required / *Type*: str /
  State of swith to set (on/off).

Returns:

/ *Type*: int /
Return ret code, 1 for succeed, 0 for failure.

21.1.3 Method: notify_updates

Notify updates to the realtime update channel for Cleware devices.

Returns:

(*no returns*)

Chapter 22

ServiceLogger.py

22.1 Class: ServiceLogger

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import ServiceLogger
```

Logger class. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-servicelogger-servicelogger-set-logger ----------

22.1.1 Method: get_config_file

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

22.1.2 Method: log

22.1.3 Method: log_info

22.1.4 Method: log_debug

22.1.5 Method: log_warning

22.1.6 Method: log_error

22.1.7 Method: log_critical

Chapter 23

Utils.py

23.1 Class: Singleton

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import Singleton
```

Class to implement Singleton Design Pattern. This class is used to derive the TTFisClientReal as only a single instance of this class is allowed.

Disabled pyLint Messages: R0903: Too few public methods (%s/%s) Used when class has too few public methods, so be sure it's really worth it.

> This base class implements the Singleton Design Pattern required for the TTFisClientReal. Adding further methods does not make sense.

23.2 Class: Utils

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import Utils
```

Class to implement utilities for supporting development. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-utils-utils-get-all-descendant-classes ----------

Get all descendant classes of a class Args: cls: Input class for finding descendants.

Returns: Array of descendant classes.

23.2.1 Method: get_all_sub_classes

Get all children classes of a class Args: cls: Input class for finding children.

Returns: Array of children classes.

23.2.2 Method: caller_name

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

23.2.3 Method: load_library

Load native library depend on the calling convention. Args: path: library path. is_stdcall: determine if the library's calling convention is stdcall or cdecl.

Returns: Loaded library object.

23.2.4 Method: is_ascii_or_unicode

Check if the string is ascii or unicode Args: str_check: string for checking codecs: encoding type list

Returns: True : if checked string is ascii or unicode False : if checked string is not ascii or unicode

23.2.5 Method: get_registry_parameters

Get service's parameters which stored in registry. Args: service_name: Name of the service. key_name: Name of register key.

Returns: Value of service's parameters

23.2.6 Method: create_registry_parameters

Create service's parameters and store it to registry. Args: service_name: Name of the service. key_name: Name of register key. parameters: Parameters to be stored.

Returns: None

23.3 Class: Job

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import Job
```

23.3.1 Method: stop

23.3.2 Method: run

Chapter 24

__main__.py

24.1 Function: main

24.2 Function: signal_handler

Chapter 25

main.py

25.1 Function: main

25.2 Function: signal_handler

Chapter 26

start_bridge.py

Standalone FastAPI UI Bridge launcher.

Starts the FastAPI bridge that exposes microservices via REST API and WebSocket. Can be spawned by the Electron GUI or run directly from the command line.

26.1 Function: parse_args

26.2 Function: main

Chapter 27

start_registry.py

Standalone Service Registry launcher.

Starts the Service Registry that tracks all services, handles registration events, and broadcasts updates to UI clients via a fanout exchange.

27.1 Function: parse_args

27.2 Function: main

Chapter 28

__init__.py

biplist -- a library for reading and writing binary property list files.

Binary Property List (plist) files provide a faster and smaller serialization format for property lists on OS X. This is a library for generating binary plists which can be read by OS X, iOS, or other clients.

The API models the plistlib API, and will call through to plistlib when XML serialization or deserialization is required.

To generate plists with UID values, wrap the values with the Uid object. The value must be an int.

To generate plists with NSData/CFData values, wrap the values with the Data object. The value must be a string.

Date values can only be datetime.datetime objects.

The exceptions InvalidPlistException and NotBinaryPlistException may be thrown to indicate that the data cannot be serialized or deserialized as a binary plist.

Plist generation example:

> from biplist import * from datetime import datetime plist = {'aKey':'aValue', '0':1.322, 'now':datetime.now(), 'list':[1,2,3], 'tuple':('a','b','c') } try: writePlist(plist, "example.plist") except (InvalidPlistException, NotBinaryPlistException), e: print "Something bad happened:", e

Plist parsing example:

> from biplist import * try: plist = readPlist("example.plist") print plist except (InvalidPlistException, NotBinaryPlistException), e: print "Not a plist:", e

28.1 Function: readPlist

Raises NotBinaryPlistException, InvalidPlistException microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---wrapdataobject ==========

28.2 Function: writePlist

28.3 Function: readPlistFromString

28.4 Function: writePlistToString

28.5 Function: is_stream_binary_plist

28.6 Class: Uid

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import Uid
```

Wrapper around integers for representing UID values. This is used in keyed archiving. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---data ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import Data
```

Wrapper around bytes to distinguish Data values. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---invalidplistexception ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import InvalidPlistException
```

Raised when the plist is incorrectly formatted. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---notbinaryplistexception ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import NotBinaryPlistException
```

Raised when a binary plist was expected but not encountered. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistreader ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import PlistReader
```

28.6.1 Method: parse

28.6.2 Method: reset

28.6.3 Method: readRoot

28.6.4 Method: setCurrentOffsetToObjectNumber

28.6.5 Method: beginOffsetProtection

28.6.6 Method: endOffsetProtection

28.6.7 Method: readObject

28.6.8 Method: readContents

28.6.9 Method: readInteger

28.6.10 Method: readReal

28.6.11 Method: readRefs

28.6.12 Method: readArray

28.6.13 Method: readDict

28.6.14 Method: readAsciiString

28.6.15 Method: readUnicode

28.6.16 Method: readDate

28.6.17 Method: readData

28.6.18 Method: readUid

28.6.19 Method: getSizedInteger

Numbers of 8 bytes are signed integers when they refer to numbers, but unsigned otherwise. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---hashablewrapper ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import HashableWrapper
```

28.7 Class: BoolWrapper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import BoolWrapper
```

28.8 Class: FloatWrapper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import FloatWrapper
```

28.9 Class: StringWrapper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import StringWrapper
```

28.9.1 Method: encodingMarker

28.10 Class: PlistWriter

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import PlistWriter
```

28.10.1 Method: reset

28.10.2 Method: positionOfObjectReference

If the given object has been written already, return its position in the offset table. Otherwise, return None. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeroot ----------

Strategy is: - write header - wrap root object so everything is hashable - compute size of objects which will be written - need to do this in order to know how large the object refs will be in the list/dict/set reference lists - write objects - keep objects in writtenReferences - keep positions of object references in referencePositions - write object references with the length computed previously - computer object reference length - write object reference positions - write trailer microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-beginrecursionprotection ----------

28.10.3 Method: endRecursionProtection

28.10.4 Method: wrapRoot

28.10.5 Method: incrementByteCount

28.10.6 Method: computeOffsets

28.10.7 Method: writeObjectReference

Tries to write an object reference, adding it to the references table. Does not write the actual object bytes or set the reference position. Returns a tuple of whether the object was a new reference (True if it was, False if it already was in the reference table) and the new output. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeobject ----------

Serializes the given object to the output. Returns output. If setReferencePosition is True, will set the position the object was written. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeoffsettable ----------

Writes all of the object reference offsets. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-binaryreal ----------

28.10.8 Method: binaryInt

28.10.9 Method: intSize

Returns the number of bytes necessary to store the given integer. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-realsize ----------

Chapter 29

badge.py

29.1 Function: badge_disk_icon

Chapter 30

colors.py

30.1 Function: isAColor

30.2 Function: parseColor

30.3 Class: Color

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import Color
```

30.3.1 Method: to_rgb

30.4 Class: RGB

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import RGB
```

30.4.1 Method: to_rgb

30.5 Class: HSL

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import HSL
```

30.5.1 Method: to_rgb

30.6 Class: HWB

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import HWB
```

30.6.1 Method: to_rgb

30.7 Class: CMYK

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import CMYK
```

30.7.1 Method: to_rgb

30.8 Class: Gray

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import Gray
```

30.8.1 Method: to_rgb

30.9 Class: ColorParser

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import ColorParser
```

30.9.1 Method: skipws

30.9.2 Method: expect

30.9.3 Method: expectEnd

30.9.4 Method: getToken

30.9.5 Method: parseNumber

30.9.6 Method: parseColor

30.9.7 Method: parseRGB

30.9.8 Method: parseHSL

30.9.9 Method: parseHWB

30.9.10 Method: parseCMYK

30.9.11 Method: parseGray

30.9.12 Method: parseValue

30.9.13 Method: parseAngle

Chapter 31

core.py

31.1 Function: build_dmg

31.2 Class: DMGError

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.core
↪  import DMGError
```

Chapter 32

buddy.py

32.1 Class: BuddyError

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy
↪  import BuddyError
```

32.2 Class: Block

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy
↪  import Block
```

32.2.1 Method: close

32.2.2 Method: flush

32.2.3 Method: invalidate

32.2.4 Method: zero_fill

32.2.5 Method: tell

32.2.6 Method: seek

32.2.7 Method: read

32.2.8 Method: write

32.2.9 Method: insert

32.2.10 Method: delete

32.3 Class: Allocator

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy
↪  import Allocator
```

32.3.1 Method: open

32.3.2 Method: close

32.3.3 Method: flush

32.3.4 Method: read

Read data at ‘offset', or raise an exception. ‘size_or_format' may either be a byte count, in which case we return raw data, or a format string for ‘struct.unpack', in which case we work out the size and unpack the data before returning it. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-write ----------

Write data at ‘offset', or raise an exception. ‘data_or_format' may either be the data to write, or a format string for ‘struct.pack', in which case we pack the additional arguments and write the resulting data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-get-block ----------

32.3.5 Method: allocate

Allocate or reallocate a block such that it has space for at least ‘bytes' bytes. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-release ----------

32.3.6 Method: iterkeys

32.3.7 Method: keys

Chapter 33

store.py

33.1 Class: ILocCodec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import ILocCodec
```

33.1.1 Method: encode

33.1.2 Method: decode

33.2 Class: PlistCodec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import PlistCodec
```

33.2.1 Method: encode

33.2.2 Method: decode

33.3 Class: BookmarkCodec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import BookmarkCodec
```

33.3.1 Method: encode

33.3.2 Method: decode

33.4 Class: DSStoreEntry

Imported by:

```
from
 ↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
 ↪  import DSStoreEntry
```

Holds the data from an entry in a `.DS_Store` file. Note that this is not meant to represent the entry itself---i.e. if you change the type or value, your changes will *not* be reflected in the underlying file.

If you want to make a change, you should either use the `DSStore` object's `DSStore.insert` method (which will replace a key if it already exists), or the mapping access mode for `DSStore` (often simpler anyway). microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-read ----------

Read a `.DS_Store` entry from the containing Block microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-byte-length ----------

Compute the length of this entry, in bytes microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-write ----------

Write this entry to the specified Block microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore ==========

Imported by:

```
from
 ↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
 ↪  import DSStore
```

Python interface to a `.DS_Store` file. Works by manipulating the file on the disk---so this code will work with `.DS_Store` files for *very* large directories.

A `DSStore` object can be used as if it was a mapping, e.g.:

```
d['foobar.dat']['Iloc']
```

will fetch the "Iloc" record for "foobar.dat", or raise `KeyError` if there is no such record. If used in this manner, the `DSStore` object will return (type, value) tuples, unless the type is "blob" and the module knows how to decode it.

Currently, we know how to decode "Iloc", "bwsp", "lsvp", "lsvP" and "icvp" blobs. "Iloc" decodes to an (x, y) tuple, while the others are all decoded using `biplist`.

Assignment also works, e.g.:

```
d['foobar.dat']['note'] = ('ustr', u'Hello World!')
```

as does deletion with `del`:

```
del d['foobar.dat']['note']
```

This is usually going to be the most convenient interface, though occasionally (for instance when creating a new `.DS_Store` file) you may wish to drop down to using `DSStoreEntry` objects directly. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-open ----------

Open a `.DS_Store` file; pass either a Python file object, or a filename in the `file_or_name` argument and a file access mode in the `mode` argument. If you are creating a new file using the "w" or "w+" modes, you may also specify a list of entries with which to initialise the file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-flush ----------

Flush any dirty data back to the file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-close ----------

Flush dirty data and close the underlying file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-insert ----------

Insert `entry` (which should be a `DSStoreEntry`) into the B-Tree. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-delete ----------

Delete an item, identified by `filename` and `code` from the B-Tree. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-find ----------

Returns a generator that will iterate over matching entries in the B-Tree.

Chapter 34

alias.py

34.1 Function: encode_utf8

34.2 Function: decode_utf8

34.3 Class: AppleShareInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import AppleShareInfo
```

34.4 Class: VolumeInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import VolumeInfo
```

34.4.1 Method: filesystem_type

34.5 Class: TargetInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import TargetInfo
```

34.6 Class: Alias

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import Alias
```

34.6.1 Method: from_bytes

Construct an `Alias` object given binary Alias data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-alias-alias-for-file ----------

Create an `Alias` that points at the specified file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-alias-alias-to-bytes ----------

Returns the binary representation for this `Alias`.

Chapter 35

bookmark.py

35.1 Function: iteritems

35.2 Class: Data

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↪  import Data
```

35.3 Class: URL

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↪  import URL
```

35.3.1 Method: absolute

Return an absolute URL. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↪  import Bookmark
```

35.3.2 Method: from_bytes

Create a `Bookmark` given byte data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-get ----------

Lookup the value for a given key, returning a default if not present. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-to-bytes ----------

Convert this `Bookmark` to a byte representation. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-for-file ----------

Construct a `Bookmark` for a given file.

Chapter 36

osx.py

36.1 Function: getattrlist

36.2 Function: fgetattrlist

36.3 Function: statfs

36.4 Function: fstatfs

36.5 Class: attrlist

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import attrlist
```

36.6 Class: attribute_set_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import attribute_set_t
```

36.7 Class: fsobj_id_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import fsobj_id_t
```

36.8 Class: timespec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import timespec
```

36.9 Class: attrreference_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import attrreference_t
```

36.10 Class: fsid_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import fsid_t
```

36.11 Class: guid_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import guid_t
```

36.12 Class: kauth_ace

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import kauth_ace
```

36.13 Class: kauth_acl

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import kauth_acl
```

36.14 Class: kauth_filesec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import kauth_filesec
```

36.15 Class: diskextent

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import diskextent
```

36.16 Class: Point

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import Point
```

36.17 Class: Rect

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import Rect
```

36.18 Class: FileInfo

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import FileInfo
```

36.19 Class: FolderInfo

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import FolderInfo
```

36.20 Class: FinderInfo

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import FinderInfo
```

36.21 Class: vol_capabilities_attr_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import vol_capabilities_attr_t
```

36.22 Class: vol_attributes_attr_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import vol_attributes_attr_t
```

36.23 Class: struct_statfs

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import struct_statfs
```

Chapter 37

utils.py

37.1 Class: UTC

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.utils
↪  import UTC
```

37.1.1 Method: utcoffset

37.1.2 Method: dst

37.1.3 Method: tzname

Chapter 38

launcher.py

FastAPI Bridge launcher.

Starts the FastAPI Bridge (REST/WS gateway) that connects the MicroserviceManagerGUI to microservices via RabbitMQ.

38.1 Function: parse_args

38.2 Function: main

Chapter 39

ClewareAccessHelper.py

39.1 Class: ClewareAccessHelper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelpe
↪  import ClewareAccessHelper
```

ClewareAccessHelper class acts as a proxy class in Proxy pattern and forward the request to the real helper depend on platform. microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-import-modules-from-paths ----------

Import all modules from given paths.

Arguments:

- `paths`
  / *Condition*: required / *Type*: list /
  List of paths to import modules from.

39.1.1 Method: load_config

Load the user config port name for USB access. Returns: None microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-get-config ----------

Get Cleware ports' config. Returns: Dictionary of ports' setting. microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-set-config ----------

Save Cleware ports' config to file. Args: config_json: data to be saved.

Returns: res: saved result.

39.1.2 Method: init_cleware

39.1.3 Method: open_cleware

39.1.4 Method: close_cleware

39.1.5 Method: get_handle

39.1.6 Method: set_value

39.1.7 Method: get_value

39.1.8 Method: set_switch

39.1.9 Method: set_switch_by_port_name

39.1.10 Method: get_switch

39.1.11 Method: get_version

39.1.12 Method: get_usb_type

39.1.13 Method: get_serial_number

39.1.14 Method: valid_ser_number

39.1.15 Method: get_hw_version

39.1.16 Method: is_ampel

39.1.17 Method: iox

39.1.18 Method: get_all_devices_state

Chapter 40

ClewareAccessHelperAbs.py

40.1 Class: ClewareAccessHelperAbs

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelpe
↪  import ClewareAccessHelperAbs
```

ClewareAccessHelperAbs acts as a abstract class for the real Helper in Proxy Pattern. microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelperabs-clewareaccesshelperabs-init-cleware
---

40.1.1 Method: open_cleware

40.1.2 Method: close_cleware

40.1.3 Method: set_value

40.1.4 Method: get_value

40.1.5 Method: set_switch

40.1.6 Method: get_switch

40.1.7 Method: get_handle

40.1.8 Method: get_version

40.1.9 Method: get_usb_type

40.1.10 Method: get_usb_type

40.1.11 Method: get_serial_number

40.1.12 Method: valid_ser_number

40.1.13 Method: get_hw_version

40.1.14 Method: is_ampel

40.1.15 Method: iox

Chapter 41

ClewareAccessHelperLinux.py

41.1 Class: ClewareAccessHelperLinux

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelpe
↪  import ClewareAccessHelperLinux
```

ClewareAccessHelperLinux acts as the real object on Linux platform in Proxy Pattern

- This is the wrapper class for the functions provided by C/C++ source for Linux.
- The C/C++ source for Linux is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN
- The C/C++ source for Linux is modified by Cuong Nguyen (RBVH/ENG22) for support python to load dynamic library on Linux.

41.1.1 Method: init_cleware

41.1.2 Method: open_cleware

41.1.3 Method: close_cleware

41.1.4 Method: get_handle

41.1.5 Method: set_value

41.1.6 Method: get_value

41.1.7 Method: set_switch

41.1.8 Method: get_switch

41.1.9 Method: get_version

41.1.10 Method: get_usb_type

41.1.11 Method: get_serial_number

41.1.12 Method: valid_ser_number

41.1.13 Method: get_hw_version

41.1.14 Method: is_ampel

41.1.15 Method: iox

41.1.16 Method: get_all_sw_state

Chapter 42

ClewareAccessHelperWindows.py

42.1 Class: ClewareAccessHelperWindows

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelpe
↪  import ClewareAccessHelperWindows
```

ClewareAccessHelperWindows acts as the real object on Windows platform in Proxy Pattern

- This is the wrapper class for the functions provided by USBaccess.dll.
- The USBaccess.dll is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN

42.1.1 Method: init_cleware

42.1.2 Method: open_cleware

42.1.3 Method: close_cleware

42.1.4 Method: get_handle

42.1.5 Method: set_value

42.1.6 Method: get_value

42.1.7 Method: set_switch

42.1.8 Method: get_switch

42.1.9 Method: get_version

42.1.10 Method: get_usb_type

42.1.11 Method: get_serial_number

42.1.12 Method: valid_ser_number

42.1.13 Method: get_hw_version

42.1.14 Method: is_ampel

42.1.15 Method: iox

Chapter 43

ServiceCleware.py

43.1 Class: ServiceCleware

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ServiceCleware
↪  import ServiceCleware
```

Service for controlling Cleware devices.

This class extends ServiceBase to provide functionalities specific to managing and controlling Cleware devices.

43.1.1 Method: svc_api_get_all_devices_state

Retrieve the state of all Cleware devices.

Returns:

/ *Type*: dict /
A dictionary containing the states of all Cleware devices.

43.1.2 Method: svc_api_set_switch

Set state for a Cleware device's switch.

Arguments:

- `device_no`
  / *Condition*: required / *Type*: str /
  Cleware device's number.
- `switch_id`
  / *Condition*: required / *Type*: str /
  Switch number to turn on or off.
- `state`
  / *Condition*: required / *Type*: str /
  State of swith to set (on/off).

Returns:

/ *Type*: int /
Return ret code, 1 for succeed, 0 for failure.

43.1.3 Method: notify_updates

Notify updates to the realtime update channel for Cleware devices.

Returns:

(*no returns*)

Chapter 44

ServiceLogger.py

44.1 Class: ServiceLogger

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ServiceLogger
↪  import ServiceLogger
```

Logger class. microservicebase-microservicemanagergui-python-services-clewareservice-servicelogger-servicelogger-set-logger ----------

44.1.1 Method: get_config_file

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

44.1.2 Method: log

44.1.3 Method: log_info

44.1.4 Method: log_debug

44.1.5 Method: log_warning

44.1.6 Method: log_error

44.1.7 Method: log_critical

Chapter 45

Utils.py

45.1 Class: Singleton

Imported by:

```
from MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.Utils
↪  import Singleton
```

Class to implement Singleton Design Pattern. This class is used to derive the TTFisClientReal as only a single instance of this class is allowed.

Disabled pyLint Messages: R0903: Too few public methods (%s/%s) Used when class has too few public methods, so be sure it's really worth it.

> This base class implements the Singleton Design Pattern required for the TTFisClientReal. Adding further methods does not make sense.

45.2 Class: Utils

Imported by:

```
from MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.Utils
↪  import Utils
```

Class to implement utilities for supporting development. microservicebase-microservicemanagergui-python-services-clewareservice-utils-utils-get-all-descendant-classes ----------

Get all descendant classes of a class Args: cls: Input class for finding descendants.

Returns: Array of descendant classes.

45.2.1 Method: get_all_sub_classes

Get all children classes of a class Args: cls: Input class for finding children.

Returns: Array of children classes.

45.2.2 Method: caller_name

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

45.2.3 Method: load_library

Load native library depend on the calling convention. Args: path: library path. is_stdcall: determine if the library's calling convention is stdcall or cdecl.

Returns: Loaded library object.

45.2.4 Method: is_ascii_or_unicode

Check if the string is ascii or unicode Args: str_check: string for checking codecs: encoding type list

Returns: True : if checked string is ascii or unicode False : if checked string is not ascii or unicode

45.2.5 Method: get_registry_parameters

Get service's parameters which stored in registry. Args: service_name: Name of the service. key_name: Name of register key.

Returns: Value of service's parameters

45.2.6 Method: create_registry_parameters

Create service's parameters and store it to registry. Args: service_name: Name of the service. key_name: Name of register key. parameters: Parameters to be stored.

Returns: None

45.3 Class: Job

Imported by:

```
from MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.Utils
↪  import Job
```

45.3.1 Method: stop

45.3.2 Method: run

Chapter 46

__main__.py

46.1 Function: main

46.2 Function: signal_handler

Chapter 47

main.py

47.1 Function: main

47.2 Function: signal_handler

Chapter 48

start_bridge.py

Standalone FastAPI UI Bridge launcher.

Starts the FastAPI bridge that exposes microservices via REST API and WebSocket. Can be spawned by the Electron GUI or run directly from the command line.

48.1 Function: parse_args

48.2 Function: main

Chapter 49

start_registry.py

Standalone Service Registry launcher.

Starts the Service Registry that tracks all services, handles registration events, and broadcasts updates to UI clients via a fanout exchange.

49.1 Function: parse_args

49.2 Function: main

Chapter 50

eventbus_config.py

50.1 Class: EventBusConfig

Imported by:

```
from MicroserviceBase.adapters.config.eventbus_config import EventBusConfig
```

Configuration for EventBusClient-based adapters.

Wraps a JSONP config file path used by EventBusClient.from_config_sync().

50.1.1 Method: load_config

Load and return the parsed config as a dict.

Requires EventBusClient to be installed.

Returns:

/ *Type*: dict /

Parsed configuration dictionary.

50.1.2 Method: to_config_source

Load config and return as a dict with optional field overrides.

Useful for creating derivative clients (e.g., registry or fanout) that share connection settings but differ in exchange configuration.

Arguments:

- `overrides`

  / *Condition*: optional / *Type*: keyword arguments /

  Fields to override in the loaded config (e.g., exchange_name, exchange_handler).

Returns:

/ *Type*: dict /

Config dictionary with overrides applied.

Chapter 51

rabbitmq_config.py

51.1 Class: RabbitMQConfig

Imported by:

```
from MicroserviceBase.adapters.config.rabbitmq_config import RabbitMQConfig
```

Configuration for connecting to RabbitMQ.

51.1.1 Method: to_connection_params

Convert to pika ConnectionParameters kwargs.

Returns:

dict suitable for pika.ConnectionParameters(**kwargs).

51.1.2 Method: from_cmd_args

Parse RabbitMQ config from command-line arguments and environment variables.

Arguments:

- `cmd_args`
  / *Condition*: optional / *Type*: list /
  List of CLI arguments, or None for sys.argv.
- `service_name`
  / *Condition*: optional / *Type*: str / *Default*: 'Service' /
  Name of the service (used in help text).

Returns:

RabbitMQConfig instance.

Chapter 52

local_hub_manager.py

Local Hub Manager -- manages a local ProcessHub instance lifecycle.

Wraps ProcessHub APIs (lazy import) so the MicroserviceManager GUI can start/stop a ProcessHub on the local machine, manage processes, and optionally join a fleet as an agent.

52.1 Class: LocalHubManager

Imported by:

```
from MicroserviceBase.adapters.local_hub.local_hub_manager import LocalHubManager
```

Manages a local ProcessHub instance lifecycle.

52.1.1 Method: start_hub

Start a local ProcessHub server.

Arguments:

- `mode`
  / *Condition*: optional / *Type*: str / *Default*: 'standalone' /
  "standalone", "agent" (join fleet), or "orchestrator".
- `xpub_port`
  / *Condition*: optional / *Type*: int / *Default*: 5555 /
  ZMQ XPUB port for the broker.
- `xsub_port`
  / *Condition*: optional / *Type*: int / *Default*: 5556 /
  ZMQ XSUB port for the broker.
- `orchestrator_url`
  / *Condition*: optional / *Type*: str /
  Fleet orchestrator ZMQ address (agent mode).
- `hub_id`
  / *Condition*: optional / *Type*: str /
  Hub identifier (agent/orchestrator mode).
- `hub_name`
  / *Condition*: optional / *Type*: str /
  Human-readable hub name (agent/orchestrator mode).

- `process_config`
  / *Condition*: optional / *Type*: dict /
  Dict of {name: config} for processes.
- `fleet_api_port`
  / *Condition*: optional / *Type*: int / *Default*: 2510 /
  FleetWebAPI port (orchestrator mode).
- `health_timeout`
  / *Condition*: optional / *Type*: float / *Default*: 30.0 /
  Health check timeout in seconds (orchestrator mode).

Returns:

/ *Type*: dict /
Status dict.

52.1.2 Method: stop_hub

Stop the local hub.

52.1.3 Method: get_status

Get current hub status.

52.1.4 Method: start_processes

Start named processes.
Arguments:

- `names`
  / *Condition*: required / *Type*: list /
  List of process names to start.

52.1.5 Method: stop_processes

Stop named processes.
Arguments:

- `names`
  / *Condition*: required / *Type*: list /
  List of process names to stop.
- `force`
  / *Condition*: optional / *Type*: bool / *Default*: False /
  If True, force-kill the processes.

52.1.6 Method: get_config

Get all process configurations.

52.1.7 Method: add_config

Add a new process configuration.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.
- `config`
  / *Condition*: required / *Type*: dict /
  Process configuration dict.

52.1.8 Method: update_config

Update an existing process configuration.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.
- `config`
  / *Condition*: required / *Type*: dict /
  Updated process configuration dict.

52.1.9 Method: remove_config

Remove a process configuration.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.

52.1.10 Method: remove_service

Remove a service -- delete config and managed service folder.

Only deletes the folder if it lives inside the managed `<config_dir>/services/` directory (never touches external paths). Stops the process first if it is still running.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Service name.

52.1.11 Method: get_service_log

Read the last *tail* lines of a process log file.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.

- `tail`
  / *Condition*: optional / *Type*: int / *Default*: 100 /
  Number of lines to read from the tail.

52.1.12 Method: get_services_dir

Return absolute path to `<config_dir>/services/`, creating it if needed.

52.1.13 Method: import_service

Import a microservice from a folder or base64-encoded ZIP.

Validates structure, copies files into `<config_dir>/services/{name}/`, and creates a hub config entry using the `${python}` placeholder.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Service name.
- `source_path`
  / *Condition*: optional / *Type*: str /
  Path to the source directory to import from.
- `zip_data`
  / *Condition*: optional / *Type*: str /
  Base64-encoded ZIP data to import from.
- `wait_time`
  / *Condition*: optional / *Type*: float / *Default*: 1.0 /
  Seconds to wait after starting before checking process health.

Returns:

/ *Type*: dict /
`{"success": bool, "message": str, "warnings": [...]}`.

52.1.14 Method: reset

Reset the hub (stop processes, clear connections).

Chapter 53

service_executor.py

Service-aware process executor.

Wraps a `SimpleExecutor` and attempts graceful RPC shutdown (`svc_api_shutdown`) before falling back to signal-based stop.

53.1 Class: ServiceExecutor

Imported by:

```
from MicroserviceBase.adapters.local_hub.service_executor import ServiceExecutor
```

Process executor that sends an RPC shutdown to MicroserviceBase services.

Delegates all operations to a wrapped `SimpleExecutor`. On `stop()`, it first tries to send `svc_api_shutdown` via RabbitMQ to the service's queue. If the service exits within `shutdown_timeout` seconds the process is considered stopped. Otherwise it falls back to the delegate's signal-based stop (CTRL_BREAK_EVENT on Windows, SIGTERM on Linux).

53.1.1 Method: start

Start a named process using the given config.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.
- `config`
  / *Condition*: required / *Type*: dict /
  Process configuration dict with 'script', 'args', 'cwd', etc.

53.1.2 Method: get_log_path

Public accessor for log file path.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.

53.1.3 Method: read_log

Public method to read process log. Returns the text content.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.
- `tail`
  / *Condition*: optional / *Type*: int / *Default*: 100 /
  Number of lines to read from the tail.

53.1.4 Method: is_running

Check if a named process is running.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.

53.1.5 Method: get_pid

Get the PID of a named process.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.

53.1.6 Method: stop

Stop a named process, attempting RPC shutdown first.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.
- `force`
  / *Condition*: optional / *Type*: bool / *Default*: False /
  If True, skip graceful shutdown and force-kill.

Chapter 54

nomad_client.py

54.1 Class: NomadAPIError

Imported by:

```
from MicroserviceBase.adapters.nomad_hub.nomad_client import NomadAPIError
```

Raised when the Nomad API returns a non-2xx response. microservicebase-adapters-nomad-hub-nomad-client-nomadclient ==========

Imported by:

```
from MicroserviceBase.adapters.nomad_hub.nomad_client import NomadClient
```

HTTP client for the HashiCorp Nomad v1 API.

Uses only `urllib` from the standard library — no external dependencies. Supports ACL token authentication and blocking queries (long-poll).

54.1.1 Method: agent_self

GET /v1/agent/self — verify agent connectivity. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-list-jobs ----------

GET /v1/jobs — list all jobs (optional prefix filter). microservicebase-adapters-nomad-hub-nomad-client-nomadclient-get-job ----------

GET /v1/job/:id — read full job specification. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-register-job ----------

POST /v1/jobs — register (create or update) a job. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-stop-job ----------

DELETE /v1/job/:id — stop a job. purge=True removes it entirely. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-get-allocations ----------

GET /v1/job/:id/allocations — list allocations for a job. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-get-allocation ----------

GET /v1/allocation/:id — read a single allocation. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-get-logs ----------

GET /v1/client/fs/logs/:alloc_id — read task logs.

Returns plain text when `plain=True`, otherwise JSON frames.

54.1.2 Method: list_jobs_blocking

GET /v1/jobs with blocking query.

Blocks until the job list changes or `wait` expires. Uses the `X-Nomad-Index` from the previous call.

Chapter 55

nomad_hub_adapter.py

55.1 Class: NomadHubAdapter

Imported by:

```
from MicroserviceBase.adapters.nomad_hub.nomad_hub_adapter import NomadHubAdapter
```

HubManagerPort implementation that manages services as Nomad jobs.

Each service config is translated into a Nomad job spec using the `raw_exec` driver (Python processes, no Docker required).

55.1.1 Method: start_hub

55.1.2 Method: stop_hub

55.1.3 Method: get_status

55.1.4 Method: start_processes

55.1.5 Method: stop_processes

55.1.6 Method: get_config

55.1.7 Method: add_config

55.1.8 Method: update_config

55.1.9 Method: remove_config

55.1.10 Method: get_service_log

55.1.11 Method: import_service

55.1.12 Method: remove_service

55.1.13 Method: reset

Chapter 56

amqp_registry_adapter.py

56.1 Class: AMQPRegistryAdapter

Imported by:

```
from MicroserviceBase.adapters.registry.amqp_registry_adapter import
↪  AMQPRegistryAdapter
```

AMQP implementation of the ServiceRegistryPort interface.
Uses RabbitMQ topic exchange for service registration events and fanout exchange for realtime update broadcasts.

56.1.1 Method: register

Register a service by publishing to the service_information exchange.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

56.1.2 Method: unregister

Unregister a service by publishing to the service_information exchange.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

56.1.3 Method: subscribe_to_events

Subscribe to service registration events.
Runs in the calling thread (blocking). Typically called from a daemon thread.
Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, properties, body).

56.1.4 Method: notify_update

Broadcast services update to the realtime update fanout exchange.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  dict of all current service information.

56.1.5 Method: subscribe_to_updates

Subscribe to the realtime update fanout exchange.

Unlike `subscribe_to_events` (which listens for individual on/off events), this subscribes to the full services dict broadcast by the Registry after each change. Uses an exclusive temporary queue to avoid competing consumers.

Runs in the calling thread (blocking). Typically called from a daemon thread.

Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(services_info_dict) called with the full services dict.

56.1.6 Method: get_update_channel_name

Return the realtime update exchange name.

56.1.7 Method: cleanup_update_exchange

Delete the realtime update exchange (for cleanup on shutdown).

56.1.8 Method: cleanup

Close event subscription resources and clean up the update exchange.

Chapter 57

eventbus_registry_adapter.py

57.1 Class: EventBusRegistryAdapter

Imported by:

```
from MicroserviceBase.adapters.registry.eventbus_registry_adapter import
↪  EventBusRegistryAdapter
```

EventBusClient implementation of the ServiceRegistryPort interface.

Uses EventBusClient with TopicExchangeHandler for service registration events and a FanoutExchangeHandler for realtime update broadcasts.

Both clients are created from the same JSONP config with exchange overrides via `EventBusConfig.to_config_source()`.

57.1.1 Method: register

Register a service by publishing to the service_information exchange.

Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

57.1.2 Method: unregister

Unregister a service by publishing to the service_information exchange.

Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

57.1.3 Method: subscribe_to_events

Subscribe to service registration events.

Blocks the calling thread. Typically called from a daemon thread. The handler receives the pika-style callback signature (ch, method, properties, body) for backward compatibility.

Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, properties, body).

57.1.4 Method: notify_update

Broadcast services update to the realtime update fanout exchange.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  dict of all current service information.

57.1.5 Method: get_update_channel_name

Return the realtime update exchange name.

57.1.6 Method: cleanup

Close all EventBusClient connections.

Chapter 58

eventbus_adapter.py

58.1 Class: _ChannelProxy

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _ChannelProxy
```

Lightweight proxy that mimics a pika channel for the domain handler.
Translates basic_publish and basic_ack calls into EventBusClient operations.

58.1.1 Method: basic_publish

Publish a reply message via the EventBusClient.
Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name (ignored, EventBusClient uses its configured exchange).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the reply (the reply_to value).
- `properties`
  / *Condition*: optional / *Type*: object /
  Properties object with correlation_id attribute.
- `body`
  / *Condition*: optional / *Type*: str or bytes /
  The message body (JSON string or bytes).

58.1.2 Method: basic_ack

Acknowledge a message (no-op for EventBusClient which auto-acks).

58.2 Class: _MethodProxy

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _MethodProxy
```

Lightweight proxy that mimics a pika method frame.

58.3 Class: _PropertiesProxy

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _PropertiesProxy
```

Lightweight proxy that mimics pika BasicProperties.

58.4 Class: EventBusTransportAdapter

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import
↪  EventBusTransportAdapter
```

EventBusClient implementation of the TransportPort interface.

Creates the underlying EventBusClient from a JSONP configuration file via `EventBusClient.from_config_sync()`.

58.4.1 Method: connect

Create the EventBusClient from the JSONP config and establish connection.

58.4.2 Method: disconnect

Close the EventBusClient connection.

58.4.3 Method: consume

Start consuming RPC requests for a service.

Subscribes to the routing key and blocks the calling thread. Incoming messages are adapted to the pika-style handler signature.

Arguments:

- `service_name`
  / *Condition*: required / *Type*: str /
  Name used as the service identifier.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for message binding.
- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name (must match the configured exchange).
- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, props, body).

58.4.4 Method: rpc_call

Send an RPC request and wait for the response.

Creates a temporary subscription for the reply, sends the request with reply_to and correlation_id headers, and waits for the response.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  dict to send as the request body.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  Exchange to publish to (must match configured exchange).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the target service.

Returns:

Parsed response dict.

58.4.5 Method: publish

Publish a message to the exchange.

Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name (must match configured exchange).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key.
- `body`
  / *Condition*: required / *Type*: str, bytes, or dict /
  Message body.
- `properties`
  / *Condition*: optional / *Type*: dict /
  Optional message properties (used for headers).

Chapter 59

rabbitmq_adapter.py

59.1 Class: RabbitMQTransportAdapter

Imported by:

```
from MicroserviceBase.adapters.transport.rabbitmq_adapter import
↪  RabbitMQTransportAdapter
```

RabbitMQ implementation of the TransportPort interface.

59.1.1 Method: connect

Establish connection to RabbitMQ.

59.1.2 Method: disconnect

Close connection to RabbitMQ.

59.1.3 Method: connection

Access the underlying pika connection (for backward compat).

59.1.4 Method: stop_consuming

Signal the consume loop to exit.

59.1.5 Method: consume

Start consuming RPC requests for a service.

Sets up the exchange, queue, binding, and starts blocking consumption.

Arguments:

- `service_name`
  / *Condition*: required / *Type*: str /
  Name used as the queue name.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for queue binding.

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name to bind to.
- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, props, body).

59.1.6 Method: rpc_call

Send an RPC request and wait for the response.
Creates a new connection for the RPC call (matching original behavior).
Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  dict to serialize as the request body.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  Exchange to publish to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the target service.
- `timeout`
  / *Condition*: optional / *Type*: int / *Default*: 30 /
  Timeout in seconds for waiting for the response.

Returns:

Parsed response dict.

59.1.7 Method: publish

Publish a message to an exchange.
Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key.
- `body`
  / *Condition*: required / *Type*: str or bytes /
  Message body (str or bytes).
- `properties`
  / *Condition*: optional / *Type*: pika.BasicProperties /
  Optional pika.BasicProperties.

Chapter 60

fastapi_bridge.py

60.1 Class: FastAPIBridge

Imported by:

```
from MicroserviceBase.adapters.ui_bridge.fastapi_bridge import FastAPIBridge
```

FastAPI implementation of UIBridgePort.

Exposes a REST API that any UI client (Electron, browser, mobile) can call. Also provides a WebSocket endpoint for push-based service updates.

Endpoints: POST /api/request - Route a service request through the transport GET /api/services - Get current services info WS /ws/updates - Real-time service update stream

Requires `fastapi` and `uvicorn` (optional dependencies).

60.1.1 Method: start

Start the FastAPI server in a background thread.

Arguments:

- `request_handler`
  / *Condition*: required / *Type*: callable /
  Callable(request_data, exchange, routing_key) -> response dict.
- `services_info_provider`
  / *Condition*: required / *Type*: callable /
  Callable() -> services info dict.

60.1.2 Method: wait

Block the calling thread until the server thread exits.

Uses a loop with timeout so that KeyboardInterrupt can be caught on Windows.

60.1.3 Method: stop

Stop the FastAPI server.

60.1.4 Method: broadcast_update

Push a services update to all connected WebSocket clients.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  dict of all current service information.

Chapter 61

exceptions.py

61.1 Class: ServiceError

Imported by:

```
from MicroserviceBase.domain.exceptions import ServiceError
```

Base exception for service-related errors.

61.2 Class: TransportError

Imported by:

```
from MicroserviceBase.domain.exceptions import TransportError
```

Raised when a transport-level operation fails.

61.3 Class: ServiceNotFoundError

Imported by:

```
from MicroserviceBase.domain.exceptions import ServiceNotFoundError
```

Raised when a requested service is not found in the registry.

61.4 Class: MethodNotFoundError

Imported by:

```
from MicroserviceBase.domain.exceptions import MethodNotFoundError
```

Raised when a requested method is not found on a service.

Chapter 62

messages.py

62.1 Class: ResultType

Imported by:

```
from MicroserviceBase.domain.messages import ResultType
```

Result types for service responses.

62.2 Class: ServiceRequest

Imported by:

```
from MicroserviceBase.domain.messages import ServiceRequest
```

Structured request to a service method.

62.2.1 Method: to_dict

Converts this `ServiceRequest` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this request.

62.2.2 Method: to_json

Converts this `ServiceRequest` instance to a JSON string.

Returns:

/ *Type*: str /
JSON string representation of this request.

62.2.3 Method: from_dict

Creates a `ServiceRequest` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing request fields.

Returns:

/ *Type*: ServiceRequest /
New instance populated from the dictionary.

62.2.4 Method: from_json

Creates a `ServiceRequest` instance from a JSON string.

Arguments:

- `json_str`
  / *Condition*: required / *Type*: str /
  JSON string containing request fields.

Returns:

/ *Type*: ServiceRequest /
New instance populated from the JSON string.

62.3 Class: ServiceResponse

Imported by:

```
from MicroserviceBase.domain.messages import ServiceResponse
```

Structured response from a service method.

62.3.1 Method: to_dict

Converts this `ServiceResponse` instance to an ordered dictionary.

Returns:

/ *Type*: OrderedDict /
Sorted ordered dictionary representation of this response.

62.3.2 Method: to_json

Converts this `ServiceResponse` instance to a JSON string.

Returns:

/ *Type*: str /
JSON string representation of this response.

62.3.3 Method: get_json

Backward-compatible alias for `to_json()`.

Returns:

/ *Type*: str /
JSON string representation of this response.

62.3.4 Method: get_data

Backward-compatible alias for `result_data`.

Returns:

/ *Type*: Any /
The result data of this response.

62.3.5 Method: from_dict

Creates a `ServiceResponse` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing response fields.

Returns:

/ *Type*: ServiceResponse /
New instance populated from the dictionary.

62.3.6 Method: from_json

Creates a `ServiceResponse` instance from a JSON string.

Arguments:

- `json_str`
  / *Condition*: required / *Type*: str /
  JSON string containing response fields.

Returns:

/ *Type*: ServiceResponse /
New instance populated from the JSON string.

62.4 Class: ServiceEvent

Imported by:

```
from MicroserviceBase.domain.messages import ServiceEvent
```

Event emitted when service state changes.

62.4.1 Method: to_dict

Converts this `ServiceEvent` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this event.

62.4.2 Method: to_json

Converts this `ServiceEvent` instance to a JSON string.

Returns:

/ *Type*: str /
JSON string representation of this event.

62.4.3 Method: from_dict

Creates a `ServiceEvent` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing event fields.

Returns:

/ *Type*: ServiceEvent /
New instance populated from the dictionary.

62.4.4 Method: from_json

Creates a `ServiceEvent` instance from a JSON string.

Arguments:

- `json_str`
  / *Condition*: required / *Type*: str /
  JSON string containing event fields.

Returns:

/ *Type*: ServiceEvent /
New instance populated from the JSON string.

Chapter 63

models.py

63.1 Class: MethodArgument

Imported by:

```
from MicroserviceBase.domain.models import MethodArgument
```

Describes a single argument of a service API method.

63.1.1 Method: to_dict

Converts this `MethodArgument` instance to a dictionary.
Returns:

/ *Type*: dict /
Dictionary representation of this argument.

63.1.2 Method: from_dict

Creates a `MethodArgument` instance from a dictionary.
Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing argument fields.

Returns:

/ *Type*: MethodArgument /
New instance populated from the dictionary.

63.2 Class: ServiceMethod

Imported by:

```
from MicroserviceBase.domain.models import ServiceMethod
```

Describes a single API method exposed by a service.

63.2.1 Method: to_dict

Converts this `ServiceMethod` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this method.

63.2.2 Method: from_dict

Creates a `ServiceMethod` instance from a name and dictionary.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  The method name.
- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing method fields.

Returns:

/ *Type*: ServiceMethod /
New instance populated from the dictionary.

63.3 Class: ServiceInfo

Imported by:

```
from MicroserviceBase.domain.models import ServiceInfo
```

Describes a service and its metadata.

63.3.1 Method: to_dict

Converts this `ServiceInfo` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this service info.

63.3.2 Method: from_dict

Creates a `ServiceInfo` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing service info fields.

Returns:

/ *Type*: ServiceInfo /
New instance populated from the dictionary.

Chapter 64

service_base.py

64.1 Class: ServiceBase

Imported by:

```
from MicroserviceBase.domain.service_base import ServiceBase
```

Pure domain base class for services.

All methods used to export APIs of a service must begin with the prefix 'svc_api'. This class handles API discovery, docstring parsing, and request dispatch. Transport and registry interactions are delegated to injected ports.

64.1.1 Method: get_service_info

Return the ServiceInfo dataclass for this service.

64.1.2 Method: get_service_info_dict

Return the service info as a plain dict (backward compat).

64.1.3 Method: serve

Start serving requests via the transport port.

64.1.4 Method: register_service

Register this service via the registry port.

64.1.5 Method: unregister_service

Unregister this service via the registry port.

64.1.6 Method: request_service

Send an RPC request to another service via the transport port.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  Dict with 'method' and 'args' keys (backward compat format).

- `exchange_name`
  / *Condition*: required / *Type*: str /
  The exchange to publish to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  The routing key for the target service.

Returns:

/ *Type*: dict /
The response dict from the target service.

64.1.7 Method: close

Close the service.

64.1.8 Method: create_request_data

Create request data for a given method name and arguments.

64.1.9 Method: get_svc_api_methods_dict

Retrieve all service API methods (methods starting with 'svc_api').

64.1.10 Method: get_svc_api_methods_info_dict

Retrieve information for all service APIs from their docstrings.

64.1.11 Method: parse_docstring

Parse function's docstring to get arguments and return information.

64.1.12 Method: svc_api_get_version

Get the service version.

Returns:

/ *Type*: str /
Version of the service.

64.1.13 Method: svc_api_get_gui_files

Compress and return GUI files as bytes.

64.1.14 Method: svc_api_get_service_files

Compress and return the entire service directory as bytes.

Returns:

/ *Type*: bytes /
ZIP archive of the service directory, or None if not downloadable.

64.1.15 Method: svc_api_get_gui_checksum

Get an MD5 checksum of all GUI files.

Returns None when gui_support is disabled or the GUIs directory is missing.

Returns:

/ *Type*: str /
MD5 hex-digest of the GUI files, or None.

64.1.16 Method: svc_api_shutdown

Gracefully shut down this service.

64.1.17 Method: is_specific_request

Check if the request is a specific request. Override in subclasses.

64.1.18 Method: on_specific_request

Handle a specific request. Override in subclasses.

Arguments:

- `request`
  / *Condition*: required / *Type*: str /
  The request method name.
- `body`
  / *Condition*: required / *Type*: dict /
  The parsed request body dict.

Returns:

/ *Type*: ServiceResponse /
A ServiceResponse, or None if not handled.

64.1.19 Method: dispatch_request

Dispatch a parsed request body and return a ServiceResponse.

This is the core domain logic: given a request dict with 'method' and 'args', find the matching API method, call it, and return a structured response.

Arguments:

- `request_body`
  / *Condition*: required / *Type*: dict /
  Dict with 'method' and 'args' keys.

Returns:

/ *Type*: ServiceResponse /
ServiceResponse with result type and data.

64.1.20 Method: on_request

Handle an incoming request (transport callback signature).

This method is called by the transport adapter. It delegates to dispatch_request for the actual domain logic, then uses the transport to send the response back.

Arguments:

- `ch`
  / *Condition*: required / *Type*: object /
  Channel object from transport.
- `method`
  / *Condition*: required / *Type*: object /
  Delivery method from transport.
- `props`
  / *Condition*: required / *Type*: object /
  Message properties from transport.
- `body`
  / *Condition*: required / *Type*: bytes /
  Raw message body (bytes or dict).

Chapter 65

service_registry.py

65.1 Class: ServiceRegistry

Imported by:

```
from MicroserviceBase.domain.service_registry import ServiceRegistry
```

Pure domain registry that tracks services, handles aliases, and routes requests.

65.1.1 Method: handle_update

Handle a service registration/unregistration event.

Arguments:

- `service_information`
  / *Condition*: required / *Type*: dict /
  Dict with 'info' and 'state' keys.

65.1.2 Method: notify_updates

Notify connected clients about service updates via the registry port.

65.1.3 Method: svc_api_shutdown

Gracefully shut down the Service Registry.

65.1.4 Method: svc_api_get_services_info

Retrieve information of all services connected to the broker.

Returns:

/ *Type*: dict /
A dictionary containing information of all connected services.

65.1.5 Method: svc_api_get_realtime_update_exchange

Retrieve the exchange name of the realtime update exchange.

Returns:

/ *Type*: str /
The name of the realtime update exchange.

65.1.6 Method: svc_api_update_alias_conf

Update the alias configuration information.

Arguments:

- `alias_string`
  / *Condition*: required / *Type*: str /
  The alias configuration string to be updated.

Returns:

(*no returns*)

65.1.7 Method: svc_api_get_alias_conf

Retrieve the alias configuration string in JSON format.

Returns:

/ *Type*: str /
The alias configuration string in JSON format.

65.1.8 Method: is_specific_request

Check if the request is an alias-routed request.

65.1.9 Method: on_specific_request

Handle an alias-routed request by forwarding to the target service.

65.1.10 Method: handle_alias_request

Handle an alias request by routing to the actual service.

Arguments:

- `body`
  / *Condition*: required / *Type*: dict /
  Dict with 'method' and 'args' keys.

Returns:

/ *Type*: ServiceResponse /
ServiceResponse or raw response dict from the target service.

65.1.11 Method: run_health_check_loop

Blocking loop for a daemon thread. Periodically checks whether each registered service still has active RabbitMQ consumers.

Arguments:

- `interval`
  / *Condition*: optional / *Type*: int / *Default*: 30 /
  Seconds between health-check cycles.
- `max_failures`
  / *Condition*: optional / *Type*: int / *Default*: 2 /
  Consecutive zero-consumer checks before auto-unregister.

- `broker_host`
  / *Condition*: optional / *Type*: str / *Default*: 'localhost' /
  RabbitMQ broker host for the temporary connection.
- `broker_port`
  / *Condition*: optional / *Type*: int / *Default*: 5672 /
  RabbitMQ broker port for the temporary connection.

65.1.12 Method: stop_health_check

Signal the health-check loop to stop.

Chapter 66

factory.py

66.1 Function: create_transport

Create a TransportPort implementation.

Arguments:

- `transport_type`
  / *Condition*: optional / *Type*: str / *Default*: 'rabbitmq' /
  Transport backend identifier. Supports 'rabbitmq' and 'eventbus'.
- `cmd_args`
  / *Condition*: optional / *Type*: list /
  Command-line arguments for config parsing (used by 'rabbitmq').
- `service_name`
  / *Condition*: optional / *Type*: str / *Default*: 'Service' /
  Service name for help text (used by 'rabbitmq').
- `config_path`
  / *Condition*: optional / *Type*: str /
  Path to JSONP config file (used by 'eventbus').

Returns:

/ *Type*: TransportPort /
A connected TransportPort instance.

66.2 Function: create_registry

Create a ServiceRegistryPort implementation.

Arguments:

- `transport_type`
  / *Condition*: optional / *Type*: str / *Default*: 'rabbitmq' /
  Transport backend identifier. Supports 'rabbitmq' and 'eventbus'.
- `cmd_args`
  / *Condition*: optional / *Type*: list /
  Command-line arguments for config parsing (used by 'rabbitmq').

- `service_name`
  / *Condition*: optional / *Type*: str / *Default*: 'Service' /
  Service name for help text (used by 'rabbitmq').
- `update_exchange_name`
  / *Condition*: optional / *Type*: str /
  Name for the realtime update fanout exchange.
- `config_path`
  / *Condition*: optional / *Type*: str /
  Path to JSONP config file (used by 'eventbus').

Returns:

/ *Type*: ServiceRegistryPort /
A ServiceRegistryPort instance.

66.3 Function: create_ui_bridge

Create a UIBridgePort implementation.

Arguments:

- `bridge_type`
  / *Condition*: optional / *Type*: str / *Default*: 'fastapi' /
  Bridge backend identifier. Currently supports 'fastapi'.
- `host`
  / *Condition*: optional / *Type*: str / *Default*: 'localhost' /
  Host to bind to.
- `port`
  / *Condition*: optional / *Type*: int / *Default*: 8000 /
  Port to bind to.

Returns:

/ *Type*: UIBridgePort /
A UIBridgePort instance (not yet started).

Chapter 67

hub_manager.py

67.1 Class: HubManagerPort

Imported by:

```
from MicroserviceBase.ports.hub_manager import HubManagerPort
```

Abstract interface for process/job orchestration backends.

Implementations manage service lifecycles through different backends: - LocalHubManager: local OS processes via ProcessHub - NomadHubAdapter: Nomad jobs via HTTP API

67.1.1 Method: start_hub

Start the orchestration backend.

Arguments:

- `mode`
  / *Condition*: optional / *Type*: str / *Default*: 'standalone' /
  Operating mode. Backend-specific values.

Returns:

- `dict` with at least {`"status": str`}

67.1.2 Method: stop_hub

Stop the orchestration backend and release resources.

Returns:

- `dict` with at least {`"status": str`}

67.1.3 Method: get_status

Get a status snapshot of all managed services/jobs.

Returns:

- `dict` containing `running`, `mode`, `processes` list, etc.

67.1.4 Method: start_processes

Start named services/jobs.

Arguments:

- `names`
  / *Condition*: required / *Type*: list[str] /
  List of service/process names to start.

Returns:

- `dict` mapping name to `{"success": bool, "message": str}`

67.1.5 Method: stop_processes

Stop named services/jobs.
Arguments:

- `names`
  / *Condition*: required / *Type*: list[str] /
  List of service/process names to stop.
- `force`
  / *Condition*: optional / *Type*: bool / *Default*: False /
  Skip graceful shutdown and force-kill immediately.

Returns:

- `dict` mapping name to `{"success": bool, "message": str}`

67.1.6 Method: get_config

Get all service/job configurations.
Returns:

- `dict` of process/job configs keyed by name.

67.1.7 Method: add_config

Add a new service/job configuration.
Arguments:

- `name`
  / *Condition*: required / *Type*: str /
- `config`
  / *Condition*: required / *Type*: dict /

Returns:

- `dict` with `{"success": bool, "message": str}`

67.1.8 Method: update_config

Update an existing service/job configuration.
Returns:

- `dict` with `{"success": bool, "message": str}`

67.1.9 Method: remove_config

Remove a service/job configuration.

Returns:

- `dict` with `{"success": bool, "message": str}`

67.1.10 Method: get_service_log

Read service/job log output.

Returns:

- `dict` with `{"name": str, "log": str}`

67.1.11 Method: import_service

Import a service package.

Returns:

- `dict` with `{"success": bool, "message": str}`

67.1.12 Method: remove_service

Remove a service entirely (stop + delete config + files).

Returns:

- `dict` with `{"success": bool, "message": str}`

67.1.13 Method: reset

Reset the hub — stop all services and clear state.

Returns:

- `dict` with `{"success": bool, "message": str}`

Chapter 68

registry.py

68.1 Class: ServiceRegistryPort

Imported by:

```
from MicroserviceBase.ports.registry import ServiceRegistryPort
```

Abstract interface for service registration and discovery.
Implementations handle how services announce themselves and how the registry discovers/tracks them.

68.1.1 Method: register

Register a service with the registry.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  Dict containing service metadata.

68.1.2 Method: unregister

Unregister a service from the registry.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  Dict containing service metadata.

68.1.3 Method: subscribe_to_events

Subscribe to service registration/unregistration events.
Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, properties, body) for events.

68.1.4 Method: notify_update

Broadcast a services update to all subscribers.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  Dict of all current service information.

68.1.5 Method: get_update_channel_name

Return the name of the realtime update channel/exchange.

Returns:

Channel/exchange name as a string.

Chapter 69

transport.py

69.1 Class: TransportPort

Imported by:

```
from MicroserviceBase.ports.transport import TransportPort
```

Abstract interface for message transport.

Implementations provide the actual connectivity (e.g., RabbitMQ, Redis, MQTT). The domain layer depends only on this interface.

69.1.1 Method: connect

Establish connection to the message broker.

69.1.2 Method: disconnect

Close connection to the message broker.

69.1.3 Method: stop_consuming

Signal the consume loop to stop.

69.1.4 Method: consume

Start consuming messages for a service.

Arguments:

- `service_name`
  / *Condition*: required / *Type*: str /
  Name of the service (used as queue name).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key to bind the queue.
- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name to bind to.
- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, props, body) for incoming messages.

69.1.5 Method: rpc_call

Send an RPC request and wait for the response.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  Dict to serialize and send as the request body.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  Exchange to publish to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the target service.

Returns:

Parsed response dict from the target service.

69.1.6 Method: publish

Publish a message to an exchange.

Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key.
- `body`
  / *Condition*: required / *Type*: str or bytes /
  Message body.
- `properties`
  / *Condition*: optional / *Type*: dict /
  Optional message properties dict.

Chapter 70

ui_bridge.py

70.1 Class: UIBridgePort

Imported by:

```
from MicroserviceBase.ports.ui_bridge import UIBridgePort
```

Abstract interface for UI communication bridge.

Implementations provide the actual UI connectivity (e.g., FastAPI REST, gRPC). Any frontend (Electron, browser, mobile) can connect through this port.

70.1.1 Method: start

Start the UI bridge server.

Arguments:

- `request_handler`
  / *Condition*: required / *Type*: callable /
  Callable that accepts a ServiceRequest dict and returns a ServiceResponse dict. Used to route UI requests to services.
- `services_info_provider`
  / *Condition*: required / *Type*: callable /
  Callable that returns the current services info dict.

70.1.2 Method: stop

Stop the UI bridge server.

70.1.3 Method: broadcast_update

Push a services update to all connected UI clients.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  Dict of all current service information.

Chapter 71

Appendix

71.1 Appendix

71.1.1 License

MicroserviceBase is licensed under the Apache License, Version 2.0.
You may obtain a copy of the License at: http://www.apache.org/licenses/LICENSE-2.0

71.1.2 References

- RabbitMQ: https://www.rabbitmq.com
- pika (RabbitMQ client for Python): https://pika.readthedocs.io
- FastAPI: https://fastapi.tiangolo.com
- Electron: https://www.electronjs.org
- Python: https://www.python.org

71.1.3 Repository

Source code: https://github.com/test-fullautomation/python-microservice-base
Issue tracker: https://github.com/test-fullautomation/python-microservice-base/issues

71.1.4 Contact

Author: Nguyen Huynh Tri Cuong
Email: Cuong.NguyenHuynhTri@vn.bosch.com

Chapter 72

History

72.1 History

72.1.1 Version 2.0.0 - 09.02.2026

Major restructure: hexagonal architecture, dual-host GUI, multi-broker support, local process hub, standalone installer.

Architecture

- **Hexagonal Architecture**: Reorganized codebase into domain, ports, and adapters layers
- **Factory Pattern**: All components created through `factory.py` for easy swapping
- **Port Interfaces**: `TransportPort`, `RegistryPort`, `UIBridgePort`
- **Adapter Implementations**: RabbitMQ transport, RabbitMQ registry, FastAPI bridge, Local Hub Manager, Service Executor

GUI v2.0

- **Dual Hosting**: Electron desktop app and browser mode via FastAPI bridge
- **Pure Web Frontend**: HTML/CSS/JS with Bootstrap 5 CDN (no Node.js dependencies)
- **Namespace**: `window.MicroserviceManager` (alias `MM`) replaces Node.js `global` and `require()` patterns
- **Service Plugins**: Dynamically loaded per-service UI components in `web/services/`
- **Dark Sidebar Theme**: Custom Bootstrap dark sidebar with accordion navigation
- **Login Modal**: Bootstrap modal replaces separate Electron popup window
- **Toast Notifications**: `MM.showToast()` replaces `notifier.notify()` and `dialog.showMessageBox()`

Multi-Broker Support

- **Connection Map**: `MM.connections` tracks per-broker state
- **Reverse Lookup**: `serviceToBroker` maps service names to broker endpoints
- **Progressive Disclosure**: Broker headers shown only with multiple connections
- **Session Persistence**: Connection state saved via `sessionStorage`

Local Process Hub

- **Hub Manager**: Start, stop, and monitor local service processes
- **Service Executor**: Process execution with `CTRL_BREAK_EVENT` for graceful shutdown on Windows
- **Config Persistence**: `hub_processes.json` with `${python}` and `${config_dir}` placeholders preserved across save operations
- **Hub Dashboard**: Real-time process status display in the GUI
- **REST API**: Full hub management via `/api/local-hub/*` endpoints

Service Management

- **Service Import**: Import microservice packages from folders or zip archives
- **Service Creator**: Interactive wizard for scaffolding new microservices
- **Registry Shutdown**: `__registry_shutdown__` sentinel notifies subscribers when a service unregisters

Electron Packaging

- **Standalone Installer**: NSIS installer for Windows (`build.bat`), Linux packages (`build.sh`)
- **Read-only Resources**: Scripts in `resources/python/`, app in `app.asar`
- **Writable User Data**: Settings, config, services, logs in `%APPDATA%/devatservgui/`
- **First-Run Init**: Template files copied from resources on first launch
- **Bridge Lifecycle**: Detached bridge process managed via PID file

Windows Process Fixes

- `SIGBREAK` handler converted to `KeyboardInterrupt` for graceful Python cleanup
- Pika `start_consuming()` replaced with `process_data_events(time_limit=1)` loop for interruptible consumption
- `subprocess.PIPE` blocking prevented by using `FileHandler` instead of `StreamHandler` for logging in subprocess mode

Documentation

- 12 Architecture Decision Records (ADR-001 through ADR-012)
- 12 PlantUML diagrams (overview, architecture, component, class, sequence, state)
- Updated README with architecture diagrams, quick start guide, and API reference
- Package documentation with Introduction, Description, and History

72.1.2 Version 0.1.0 - 02/2023

Initial version.

- Basic microservice framework with RabbitMQ communication
- Service registration and discovery
- Electron-based GUI for service monitoring

MicroserviceBase.pdf

Created at 15.03.2026 - 17:53:45

by GenPackageDoc v. 0.16.0